Tarlac State University
COLLEGE OF COMPUTER STUDIES

Simulation of CPU Scheduling Algorithms

A Case Study
Submitted to the Faculty of the College of Computer Studies
San Isidro Campus, Tarlac City,
Tarlac State University

In Partial Fulfillment of the Requirements for the
Subject Operating Systems
Academic Year 2025-2026

Submitted By:
Golingan, Reena Angela F.

Subject Instructor
Jo Anne G. Cura

Date of submission:
May 15, 2026

Table of Contents

1. Introduction 6

1.1. Overview of CPU Scheduling in Operating Systems 6

1.2. Purpose of the Case Study 6

1.3. Objectives of the Study 6

2. CPU Scheduling Algorithms 7

2.1. First-Come, First-Served (FCFS) Scheduling 7

2.1.1. Algorithm Overview 7

2.1.2. Preemptive vs Non-Preemptive Classification 7

2.1.3. Advantages and Limitations 8

2.1.4. Test Case No. 1 8

2.1.5. Test Case No. 2 9

2.2. Shortest Job First (SJF) Scheduling 10

2.2.1. Algorithm Overview 10

2.2.2. Preemptive vs Non-Preemptive Classification 10

2.2.3. Advantages and Limitations 11

2.2.4. Test Case No. 1 11

2.2.5. Test Case No. 2 12

2.3. Shortest Remaining Time (SRT) Scheduling 13

2.3.1. Algorithm Overview 13

2.3.2. Preemptive vs Non-Preemptive Classification 13

2.3.3. Advantages and Limitations 13

2.3.4. Test Case No. 1 14

2.3.5. Test Case No. 2 15

2.4. Round Robin (RR) Scheduling 16

2.4.1. Algorithm Overview 16

2.4.2. Preemptive vs Non-Preemptive Classification 16

2.4.3. Advantages and Limitations 16

2.4.4. Test Case No. 1 (Time Quantum = 2) 17

2.4.5. Test Case No. 2 (Time Quantum = 3) 18

2.5. Priority Scheduling 19

2.5.1. Algorithm Overview 19

2.5.2. Preemptive vs Non-Preemptive Classification 19

2.5.3. Advantages and Limitations 19

2.5.4. Test Case No. 1 (Non-Preemptive Priority Scheduling) 20

2.5.5. Test Case No. 2 (Preemptive Priority Scheduling) 21

2.6. Priority with Round Robin Scheduling 22

2.6.1. Algorithm Overview 22
2.6.2. Preemptive vs Non-Preemptive Classification 22
2.6.3. Advantages and Limitations 22
2.6.4. Test Case No. 1 (Non-Preemptive Priority + RR, Time Quantum = 2) 23
2.6.5. Test Case No. 2 (Preemptive Priority + RR, Time Quantum = 3) 24
3. Performance Analysis and Discussion 26
3.1. Comparative Analysis of Algorithms 26
3.1.1. Standardized Test Workload 26
3.1.2. Standardized Test Results 26
3.1.3. Analysis of Results 27
3.2. Performance Observations 27
3.3. Practical Relevance in Modern Operating Systems 28
4. Conclusion 29
4.1. Key Insights and Learnings 29
4.2. Challenges and Solutions 30
4.3. Future Improvements and Recommendations 31
5. References 32
6. Appendices 33
6.1. Repository Link 33
6.2. Executable File/ Link 33
6.3. Source Code 33

List of Figures and Tables

Figure 1. User Interface of the CPU Scheduling Algorithm Simulator 7
Figure 2. FCFS Scheduling – Test Case No. 1 Input Parameter 8
Figure 3. FCFS Scheduling – Test Case No. 1 Output 9
Figure 4. FCFS Scheduling – Test Case No. 2 Input Parameter 9
Figure 5. FCFS Scheduling – Test Case No. 2 Output 10
Figure 6. SJF Scheduling – Test Case No. 1 Input Parameter 11
Figure 7. SJF Scheduling – Test Case No. 1 Output 12
Figure 8. SJF Scheduling – Test Case No. 2 Input Parameter 12
Figure 9. SJF Scheduling – Test Case No. 2 Output 13
Figure 10. SRTF Scheduling – Test Case No. 1 Input Parameter 14
Figure 11. SRTF Scheduling – Test Case No. 1 Output 14
Figure 12. SRTF Scheduling – Test Case No. 2 Input Parameter 15
Figure 13. SRTF Scheduling – Test Case No. 2 Output 15
Figure 14. RR Scheduling – Test Case No. 1 Input Parameter 17
Figure 15. RR Scheduling – Test Case No. 1 Output 17
Figure 16. RR Scheduling – Test Case No. 2 Input Parameter 18
Figure 17. RR Scheduling – Test Case No. 2 Output 18
Figure 18. Priority Scheduling – Test Case No. 1 Input Parameter 20
Figure 19. Priority Scheduling – Test Case No. 1 Output 20
Figure 20. Priority Scheduling – Test Case No. 2 Input Parameter 21
Figure 21. Priority Scheduling – Test Case No. 2 Output 21
Figure 22. Priority with Round Robin Scheduling – Test Case No. 1 Input Parameter 24
Figure 23. Priority with Round Robin Scheduling – Test Case No. 1 Output 24
Figure 24. Priority with Round Robin Scheduling – Test Case No. 2 Input Parameter 25
Figure 25. Priority with Round Robin Scheduling – Test Case No. 2 Output 25

Table 1. Standard Test Workload - Input Parameters 26
Table 2. Standardized Test Results 26

List of Appendices

Repository Link 33
Executable File/ Link 33
Source Code 33
index.html 33
globals.css 35
main.js 39
fcfs.js 40
sjf.js 41
srtf.js 42
rr.js 44
priority.js 46
priorityRR.js 48
getInput.js 52
validateInput.js 53
renderTable.js 53
renderGantt.js 54
simulateGantt.js 55
processColors.js 57

1. Introduction

This section presents the foundational concepts, scope, and objectives of the study, providing an overview of CPU scheduling in operating systems and introducing the purpose and goals of the developed CPU scheduling simulator.

1.1. Overview of CPU Scheduling in Operating Systems

CPU scheduling is a fundamental component of modern operating systems that determines which process in the ready queue will be allocated the processor for execution. The primary objective of CPU scheduling is to maximize CPU utilization while ensuring fairness, minimizing response time, and optimizing throughput. Effective scheduling algorithms balance multiple competing goals: keeping the CPU busy, providing quick response times for interactive processes, ensuring all processes eventually execute, and maximizing the number of processes completed per unit time.

The scheduler operates as a critical intermediary between user processes and the CPU, making rapid decisions about process execution based on various criteria such as arrival time, burst time, priority levels, and time quantum values. Different scheduling algorithms employ distinct strategies, each with inherent trade-offs in terms of fairness, efficiency, starvation prevention, and context-switching overhead.

1.2. Purpose of the Case Study

This study presents a practical, web-based implementation of six fundamental CPU scheduling algorithms, providing both educational value and hands-on experience with process management concepts. The simulator enables users to visualize how different algorithms handle process scheduling through interactive Gantt charts and comprehensive performance metrics.

Implemented as a web application using HTML, CSS, and JavaScript, the simulator offers an accessible platform for understanding scheduling behavior without requiring installation of development environments or compilation tools.

1.3. Objectives of the Study

The primary objectives of this study and CPU scheduling simulator are:

1. To implement and demonstrate six core CPU scheduling algorithms with accurate simulation logic
2. To provide visual representations of process execution through interactive Gantt charts
3. To calculate and display key performance metrics including waiting time, turnaround time, and their averages

4. To enable comparative analysis of algorithm performance under identical workloads
5. To facilitate understanding of the trade-offs between different scheduling strategies
6. To create an accessible, user-friendly platform for exploring operating system concepts

2. CPU Scheduling Algorithms

Various CPU scheduling algorithms have been developed to address the challenges of process management and resource allocation in operating systems. Each algorithm applies a different strategy for selecting the next process to execute, resulting in varying performance in terms of waiting time, turnaround time, response time, and overall system efficiency.

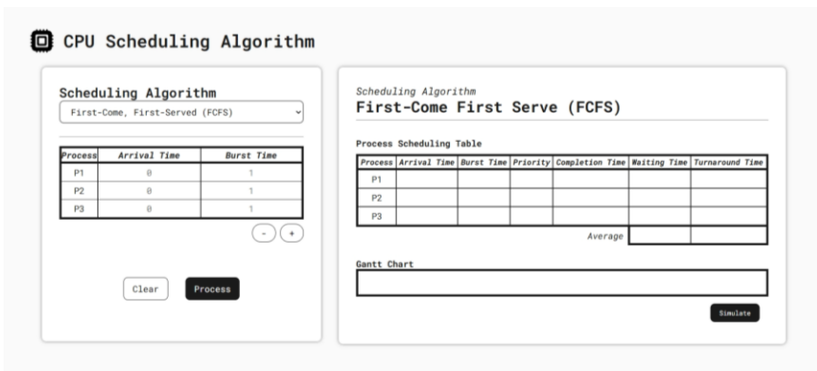


Figure 1. User Interface of the CPU Scheduling Algorithm Simulator

2.1. First-Come, First-Served (FCFS) Scheduling

2.1.1. Algorithm Overview

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm, operating on the principle that processes are executed in the exact order of their arrival in the ready queue. The process that arrives first is allocated the CPU first and runs to completion before the next process begins execution. This scheduling method is typically implemented using a First-In-First-Out (FIFO) queue structure, where newly arriving processes are appended at the end of the queue. As the CPU becomes available, it is assigned to the process at the front of the queue. Because execution strictly follows arrival order, FCFS provides a straightforward and predictable scheduling behavior that is easy to simulate and analyze.

2.1.2. Preemptive vs Non-Preemptive Classification

FCFS is a non-preemptive scheduling algorithm. Once a process begins execution, it continues until it completes or voluntarily relinquishes the CPU (e.g., for I/O operations). During execution, no other process can interrupt or preempt the currently running process, regardless of arrival time or priority. This behavior

simplifies CPU management by eliminating the need for context switching due to preemption, but it also reduces flexibility in handling dynamic or high-priority workloads.

2.1.3. Advantages and Limitations

Advantages*:*

1. Simple to understand and implement with minimal computational overhead
2. Fair in the sense that no process is starved; all processes eventually execute based on arrival order
3. No additional parameters required (no priorities, time quantum, or complex decision rules)
4. Predictable behavior, making it suitable for batch processing systems where response time is not critical

Limitations*:*

1. Suffers from the "convoy effect," where short processes are forced to wait behind long processes
2. Can result in poor average waiting time and turnaround time, especially when long processes arrive first
3. Not suitable for time-sharing or interactive systems that require quick responsiveness
4. CPU-bound processes may monopolize the processor for extended periods, reducing overall system responsiveness

2.1.4. Test Case No. 1

The following test case demonstrates the execution of the First-Come, First-Served (FCFS) scheduling algorithm using a set of processes with staggered arrival times and varying burst durations. The system was evaluated to observe how FCFS handles sequential process execution based strictly on arrival order, without preemption or prioritization.

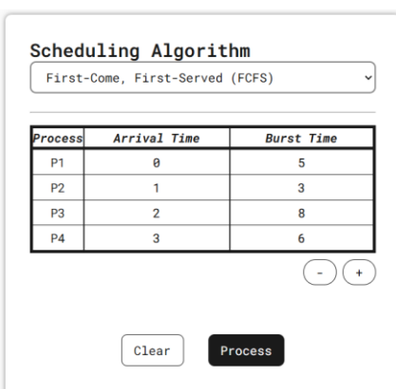


Figure 2. FCFS Scheduling – Test Case No. 1 Input Parameter

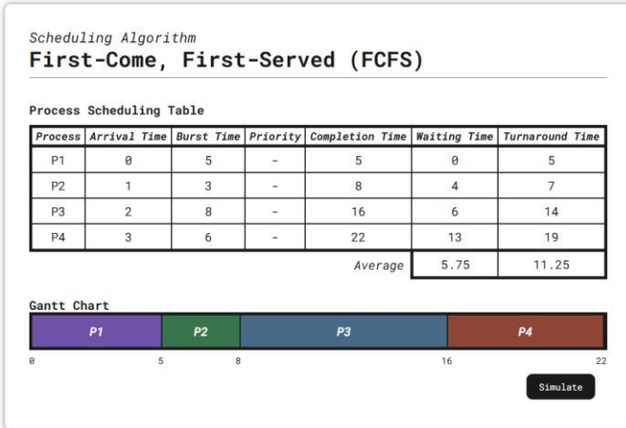


Figure 3. FCFS Scheduling – Test Case No. 1 Output

The results show that processes are executed strictly in the order of arrival, where earlier processes are completed before later ones regardless of burst time differences. As a result, shorter processes that arrive later experience increased waiting time due to the execution of earlier longer processes. The computed performance metrics yield an average waiting time of 5.75 ms and an average turnaround time of 11.25 ms, reflecting moderate efficiency but noticeable delay for later-arriving processes.

2.1.5. Test Case No. 2

This test case evaluates FCFS under a scenario where all processes arrive at time zero but have significantly different burst times. This configuration is used to further analyze the convoy effect, where long processes can delay the execution of shorter ones even when all processes are available at the same time.

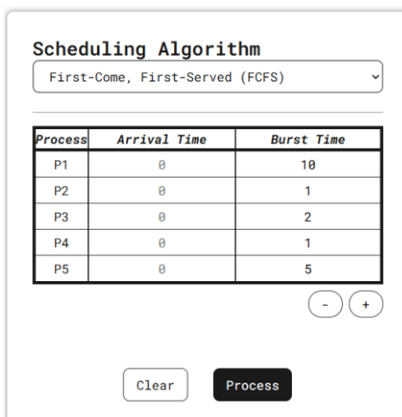
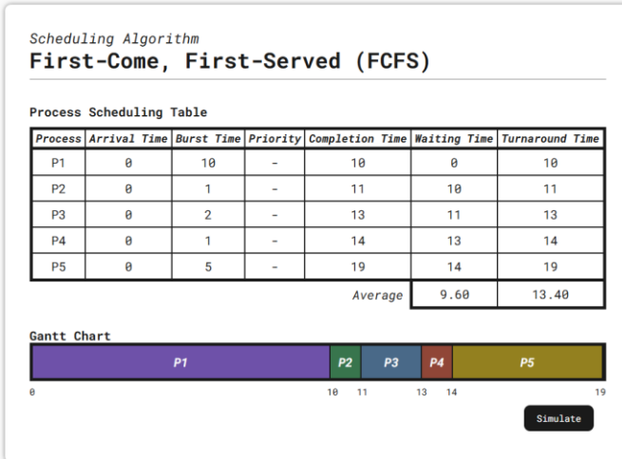


Figure 4. FCFS Scheduling – Test Case No. 2 Input Parameter



Scheduling Algorithm

First-Come, First-Served (FCFS)

Process Scheduling Table

Process	Arrival Time	Burst Time	Priority	Completion Time	Waiting Time	Turnaround Time
P1	0	10	-	10	0	10
P2	0	1	-	11	10	11
P3	0	2	-	13	11	13
P4	0	1	-	14	13	14
P5	0	5	-	19	14	19
				Average	9.60	13.40

Gantt Chart

Figure 5. FCFS Scheduling – Test Case No. 2 Output

The results clearly demonstrate the convoy effect, where the longest process (P1) executes first and causes all shorter processes to wait significantly before execution. This leads to increased waiting times for smaller processes despite their short burst durations. The average waiting time is 9.6 ms, while the average turnaround time is 13.4 ms, indicating reduced efficiency under workload conditions with uneven burst distribution.

This test case highlights the inherent limitation of FCFS scheduling, particularly its inability to optimize execution order based on process length or priority, resulting in higher waiting times in mixed workload scenarios.

2.2. Shortest Job First (SJF) Scheduling

2.2.1. Algorithm Overview

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the smallest CPU burst time for execution. The main idea is to reduce waiting time by prioritizing shorter processes, which are executed before longer ones whenever the CPU becomes available. In a non-preemptive implementation, the selected process runs until completion, while in general scheduling theory, SJF is known for its efficiency in minimizing average waiting time when burst times are accurately known or estimated. However, in practical scenarios, exact burst time prediction is difficult, which makes the algorithm more theoretical in nature than fully practical in some systems.

2.2.2. Preemptive vs Non-Preemptive Classification

SJF can be implemented in both non-preemptive and preemptive forms. In its non-preemptive form, once a process starts execution, it runs until completion. In contrast, its preemptive version (Shortest Remaining Time First) allows interruption if a newly arrived process has a shorter remaining execution time.

2.2.3. Advantages and Limitations

SJF is optimal in terms of minimizing average waiting time under ideal conditions, making it highly efficient when process burst times are known in advance. However, its major limitation is the difficulty of accurately predicting burst times in real systems. Additionally, SJF may cause starvation, where longer processes are continuously delayed if shorter processes keep arriving.

Advantages*:*

1. Provably optimal for minimizing average waiting time among non-preemptive scheduling algorithms
2. Reduces average turnaround time compared to FCFS
3. Maximizes throughput by completing more short processes quickly
4. Efficient for batch systems where process burst times are known in advance

Limitations*:*

1. Requires prior knowledge or accurate prediction of burst times, which is often impractical in real systems
2. Can cause starvation, where longer processes are indefinitely delayed if shorter processes continuously arrive
3. Difficult to implement in interactive systems with unpredictable workloads
4. May lead to unfairness due to the prolonged waiting time of long processes

2.2.4. Test Case No. 1

The following test case demonstrates the execution of the Shortest Job First (SJF) scheduling algorithm using a set of processes with varying arrival times and burst durations. This scenario was designed to evaluate how SJF prioritizes processes with shorter execution times when selecting the next process from the ready queue.

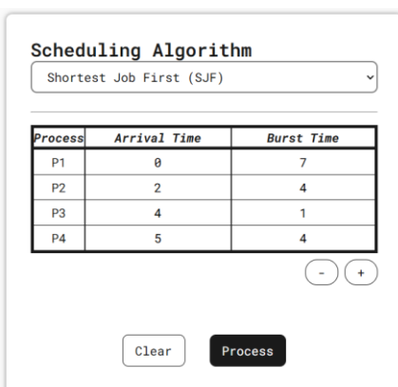


Figure 6. SJF Scheduling – Test Case No. 1 Input Parameter

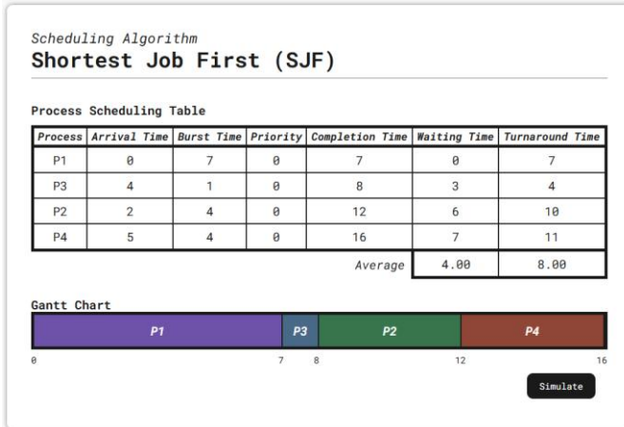


Figure 7. SJF Scheduling – Test Case No. 1 Output

The results show that the algorithm selects processes based on the shortest available burst time among the processes that have already arrived. This leads to improved scheduling efficiency, particularly when shorter processes become available during execution. However, longer processes may experience increased waiting time depending on their arrival position in the queue. The computed performance metrics yield an average waiting time of 4.0 ms and an average turnaround time of 8.0 ms, indicating efficient scheduling under this workload.

2.2.5. Test Case No. 2

This test case evaluates SJF under a workload where all processes have different arrival times and relatively larger burst durations. This configuration is used to analyze how SJF behaves when shorter processes are interleaved with longer ones during execution.

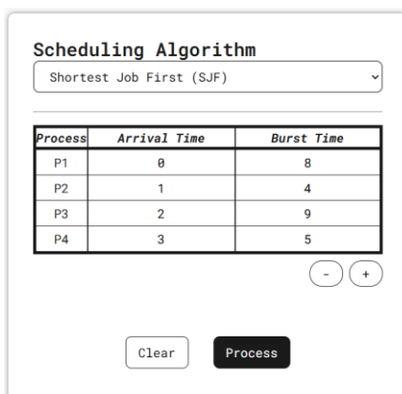
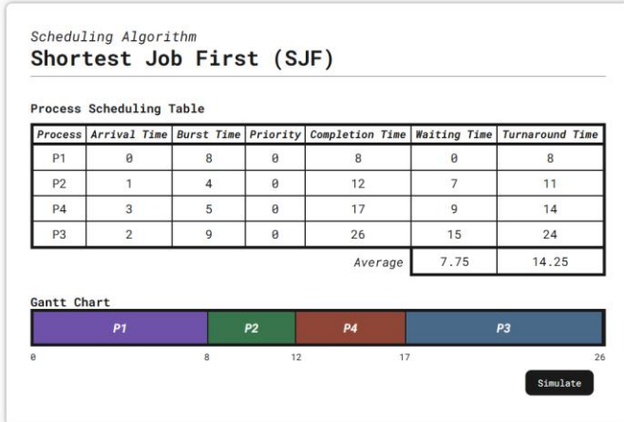


Figure 8. SJF Scheduling – Test Case No. 2 Input Parameter



Scheduling Algorithm

Shortest Job First (SJF)

Process Scheduling Table

Process	Arrival Time	Burst Time	Priority	Completion Time	Waiting Time	Turnaround Time
P1	0	8	0	8	0	8
P2	1	4	0	12	7	11
P4	3	5	0	17	9	14
P3	2	9	0	26	15	24
				Average	7.75	14.25

Gantt Chart

Figure 9. SJF Scheduling – Test Case No. 2 Output

The results show that SJF consistently selects the shortest available process at each scheduling decision point, which improves overall efficiency compared to FCFS. However, processes with longer burst times experience increased waiting time as shorter processes are prioritized whenever they become available. The average waiting time is 7.75 ms, while the average turnaround time is 14.25 ms, reflecting moderate efficiency with some delay for longer processes.

2.3. Shortest Remaining Time (SRT) Scheduling

2.3.1. Algorithm Overview

Shortest Remaining Time First (SRTF) is the preemptive version of SJF, where the CPU is assigned to the process with the smallest remaining execution time at any given moment. If a new process arrives with a shorter remaining time than the currently running process, the CPU is immediately reallocated to the new process. This dynamic decision-making allows the system to continuously prioritize shorter remaining tasks, improving responsiveness and reducing average waiting time in many scenarios.

2.3.2. Preemptive vs Non-Preemptive Classification

SRTF is a preemptive scheduling algorithm, meaning that running processes can be interrupted whenever a higher-priority condition is met—in this case, a shorter remaining burst time. This frequent preemption makes it more responsive but also increases scheduling complexity.

2.3.3. Advantages and Limitations

SRTF provides better average waiting time and responsiveness compared to non-preemptive SJF. However, it introduces higher overhead due to frequent context switching. It may also lead to starvation, particularly for long processes, if shorter processes keep entering the system continuously.

Advantages*:*

1. Provides optimal average waiting time among scheduling algorithms
2. Improves response time, especially for short processes, compared to non-preemptive SJF
3. Dynamically adapts to newly arriving processes through preemption
4. Maximizes CPU utilization and overall system throughput

Limitations*:*

1. Requires accurate prediction or estimation of remaining burst times
2. Introduces high context-switching overhead due to frequent preemptions
3. May cause starvation for long processes when shorter processes continuously arrive
4. More complex to implement due to continuous monitoring and scheduling decisions

2.3.4. Test Case No. 1

The following test case demonstrates the execution of the Shortest Remaining Time First (SRTF) scheduling algorithm using a set of processes with varying arrival times and burst durations. This scenario highlights the preemptive nature of SRTF, where the currently running process may be interrupted if a newly arrived process has a shorter remaining execution time.

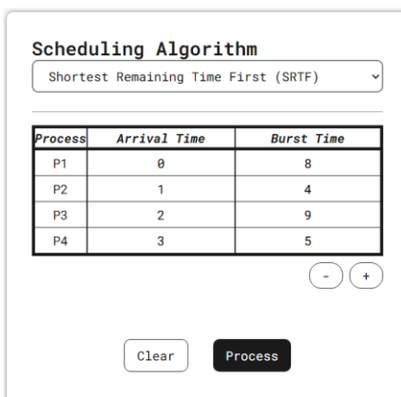


Figure 10. SRTF Scheduling – Test Case No. 1 Input Parameter

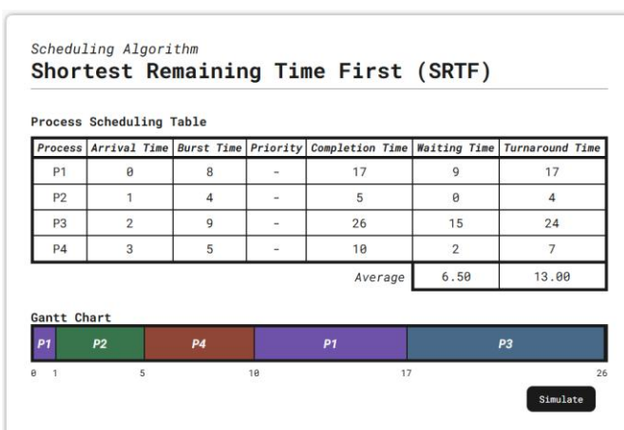


Figure 11. SRTF Scheduling – Test Case No. 1 Output

The results show that SRTF continuously evaluates the remaining burst times of all available processes and preempts the currently executing process when a shorter remaining job arrives. This behavior allows shorter processes to complete earlier, improving responsiveness. However, longer processes experience increased waiting times due to repeated preemptions. The computed performance metrics yield an average waiting time of 6.5 ms and an average turnaround time of 13.0 ms, demonstrating improved efficiency compared to non-preemptive scheduling under the same workload.

2.3.5. Test Case No. 2

This test case evaluates SRTF under a workload with staggered arrivals and varying burst times. The configuration is designed to further observe how continuous preemption affects overall scheduling performance.

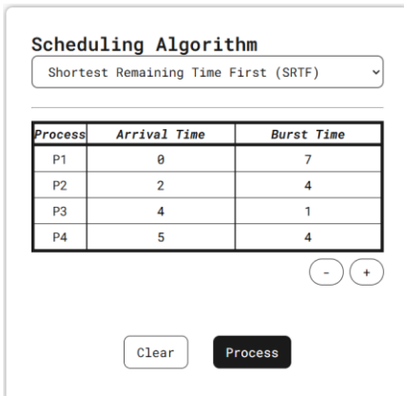


Figure 12. SRTF Scheduling – Test Case No. 2 Input Parameter

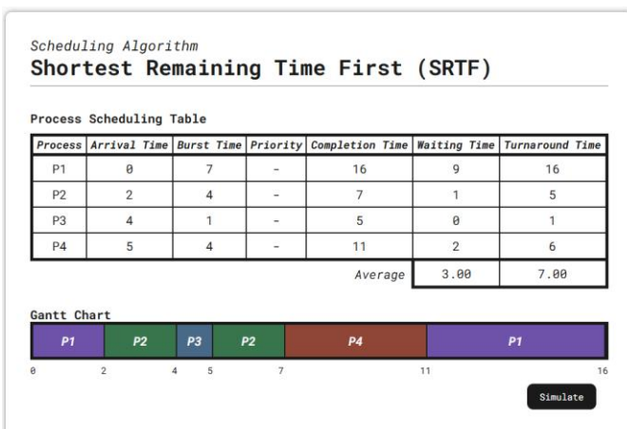


Figure 13. SRTF Scheduling – Test Case No. 2 Output

The results indicate that SRTF consistently prioritizes the process with the shortest remaining execution time at every decision point. This leads to improved performance for shorter processes, which complete earlier in the schedule. The average waiting time is 3.0 ms, while the average turnaround time is 7.0 ms, reflecting high efficiency for this workload due to effective preemptive scheduling.

The results further demonstrate that SRTF improves average waiting time compared to non-preemptive SJF by allowing continuous preemption based on remaining burst time. However, this improvement comes at the cost of increased scheduling complexity and potential overhead from frequent context switching.

2.4. Round Robin (RR) Scheduling

2.4.1. Algorithm Overview

Round Robin (RR) is a preemptive CPU scheduling algorithm designed for time-sharing systems, where each process is assigned a fixed time quantum in a cyclic order. Processes are placed in a circular queue, and each process receives CPU time for a fixed interval. If a process does not finish within its allocated time quantum, it is preempted and moved to the back of the queue. This ensures that all processes receive regular CPU access, making RR suitable for interactive and multi-user environments.

2.4.2. Preemptive vs Non-Preemptive Classification

Round Robin is a preemptive scheduling algorithm, as processes are forcibly interrupted once their time quantum expires, even if they are not yet completed. This preemption ensures fairness by preventing any single process from monopolizing the CPU.

2.4.3. Advantages and Limitations

RR is widely used in time-sharing systems because it provides fair CPU allocation and good response time for all processes. It is particularly effective for interactive applications where user responsiveness is important. However, its performance heavily depends on the size of the time quantum: a small quantum leads to excessive context switching overhead, while a large quantum reduces responsiveness and makes the system behave similarly to FCFS.

Advantages*:*

1. Ensures fairness by allocating equal CPU time to all processes
2. Provides good response time, making it suitable for interactive systems
3. Eliminates starvation, as all processes are guaranteed CPU access
4. Simple to implement and understand
5. Well-suited for time-sharing and multitasking environments

Limitations*:*

1. Performance is highly dependent on the time quantum size
2. A small-time quantum increases context-switching overhead, while a large quantum reduces responsiveness and approximates FCFS behavior

3. Higher average turnaround time compared to SJF and SRTF
4. Context switching overhead can be significant
5. Not optimal for minimizing average waiting and turnaround times

2.4.4. Test Case No. 1 (Time Quantum = 2)

The following test case demonstrates the execution of the Round Robin (RR) scheduling algorithm using a time quantum of 2. This configuration evaluates how the algorithm distributes CPU time equally among processes while ensuring fairness in execution order.

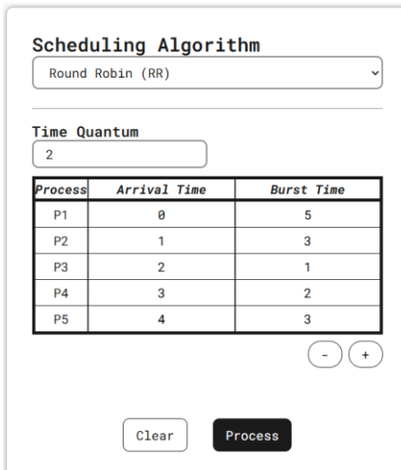


Figure 14. RR Scheduling – Test Case No. 1 Input Parameter

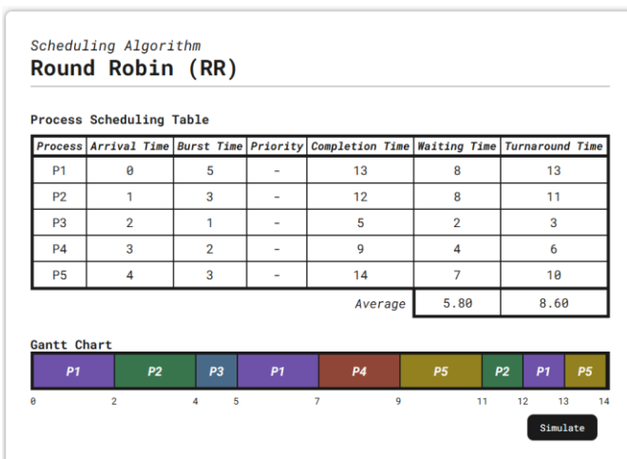


Figure 15. RR Scheduling – Test Case No. 1 Output

The results show that processes are executed in a cyclic manner, where each process receives a fixed time slice before being placed back into the ready queue if it has remaining burst time. This ensures fair CPU allocation among all processes regardless of burst duration. However, frequent context switching is observed due to the small-time quantum. The computed performance metrics yield an average waiting time of 5.8 ms and an average turnaround time of 8.6 ms, reflecting balanced but overhead-influenced scheduling behavior.

2.4.5. Test Case No. 2 (Time Quantum = 3)

This test case evaluates Round Robin scheduling using a larger time quantum of 3 to analyze its effect on system performance and context switching frequency.

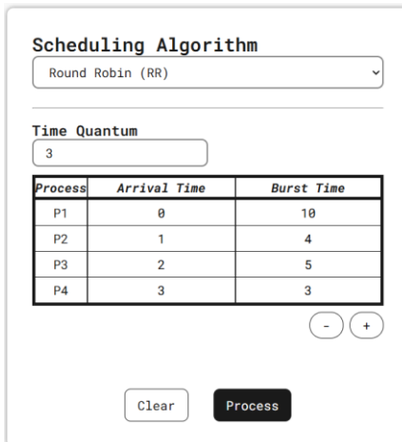


Figure 16. RR Scheduling – Test Case No. 2 Input Parameter

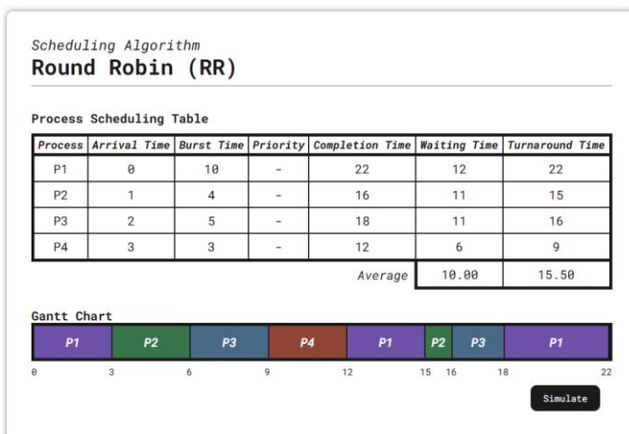


Figure 17. RR Scheduling – Test Case No. 2 Output

The results indicate that increasing the time quantum reduces the frequency of context switching, allowing processes to execute for longer uninterrupted periods. However, this also causes shorter processes to wait longer before receiving CPU time, resulting in increased average waiting time. The computed average waiting time is 10.0 ms, while the average turnaround time is 15.5 ms, showing reduced responsiveness compared to a smaller time quantum.

These results demonstrate that Round Robin scheduling is highly sensitive to time quantum selection. A smaller quantum improves fairness and responsiveness but increases context switching overhead, while a larger quantum reduces overhead but may degrade responsiveness and cause behavior closer to FCFS.

2.5. Priority Scheduling

2.5.1. Algorithm Overview

Priority Scheduling is a CPU scheduling algorithm where each process is assigned a priority value, and the CPU is allocated to the process with the highest priority among those in the ready queue. The priority can be defined based on system requirements such as process importance, resource needs, or user-defined values. In this approach, the scheduler continuously selects the process with the most favorable priority for execution whenever the CPU becomes available. This allows the system to give preference to critical tasks, ensuring that important processes are executed earlier than less critical ones.

Priority Scheduling can operate in a static or dynamic manner. In static priority systems, the priority of a process remains constant throughout its execution, while in dynamic systems, priorities may change based on factors such as waiting time or system aging.

2.5.2. Preemptive vs Non-Preemptive Classification

Priority Scheduling can be implemented in both preemptive and non-preemptive forms. In non-preemptive priority scheduling, a running process continues execution until it completes or voluntarily releases the CPU. In preemptive priority scheduling, a newly arrived process with a higher priority can immediately interrupt the currently running process and take control of the CPU. This makes the preemptive version more responsive for high-priority tasks but also increases system complexity due to more frequent context switching.

2.5.3. Advantages and Limitations

Priority Scheduling allows important or time-sensitive processes to be executed earlier, making it suitable for systems where certain tasks require immediate attention. It also provides flexibility in managing workloads based on importance rather than just arrival time or burst duration. However, its main limitation is the risk of starvation, where low-priority processes may be delayed indefinitely if higher-priority processes continue to arrive. To mitigate this issue, techniques such as aging are often used, where the priority of waiting processes gradually increases over time to ensure eventual execution.

Advantages*:*

1. Allows important or time-critical processes to execute first
2. Provides flexibility through user-defined or system-defined priority levels
3. Improves system responsiveness for high-priority or urgent tasks

4. Preemptive implementation ensures immediate attention to critical processes

Limitations*:*

1. May cause starvation, where low-priority processes are indefinitely delayed if higher-priority processes continuously arrive
2. Requires careful priority assignment to avoid unfairness and imbalance
3. Preemptive version introduces context-switching overhead
4. Often requires additional mechanisms, such as aging, to prevent starvation and ensure eventual execution

2.5.4. Test Case No. 1 (Non-Preemptive Priority Scheduling)

The following test case demonstrates the execution of the non-preemptive Priority Scheduling algorithm using a set of processes with varying arrival times, burst durations, and priority levels. This configuration evaluates how the algorithm selects processes based on priority after a process has completed execution.

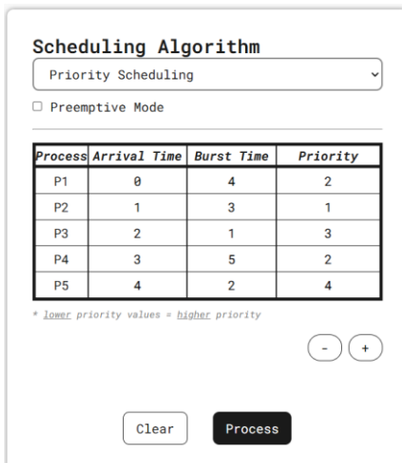


Figure 18. Priority Scheduling – Test Case No. 1 Input Parameter

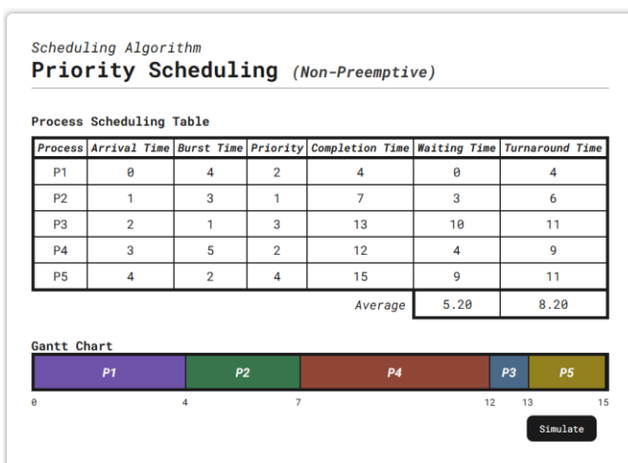


Figure 19. Priority Scheduling – Test Case No. 1 Output

The results show that once a process is selected for execution, it runs to completion without interruption, even if higher-priority processes arrive during execution. Scheduling decisions are made only when the CPU becomes idle. This leads to situations where higher-priority processes may experience waiting time if a lower-priority process is already executing. The computed performance metrics yield an average waiting time of 5.2 ms and an average turnaround time of 8.2 ms, reflecting moderate efficiency under non-preemptive execution constraints.

2.5.5. Test Case No. 2 (Preemptive Priority Scheduling)

This test case evaluates the preemptive version of Priority Scheduling, where the CPU can be reassigned if a higher-priority process arrives during execution.

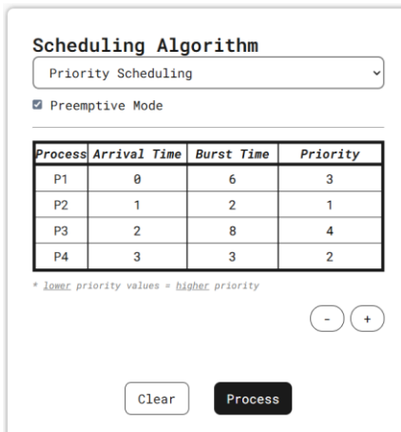


Figure 20. Priority Scheduling – Test Case No. 2 Input Parameter

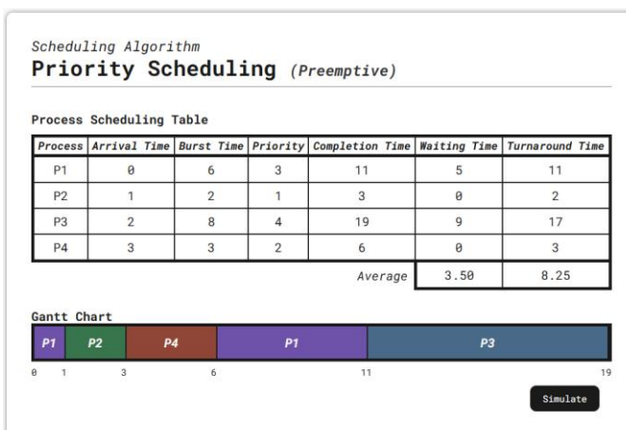


Figure 21. Priority Scheduling – Test Case No. 2 Output

The results demonstrate that the algorithm immediately preempts the currently running process when a higher-priority process becomes available. This ensures that critical tasks are executed with minimal delay. In this case, process P2 preempts P1 upon arrival due to its higher priority, improving system responsiveness. The average waiting time is 3.5 ms, while the average turnaround

time is 8.25 ms, indicating improved responsiveness compared to the non-preemptive version.

These test case highlights the key difference between non-preemptive and preemptive priority scheduling. While the non-preemptive version offers simpler execution, the preemptive version provides better responsiveness for high-priority processes at the cost of increased context-switching complexity.

2.6. Priority with Round Robin Scheduling

2.6.1. Algorithm Overview

Priority with Round Robin (Priority + RR) Scheduling is a hybrid CPU scheduling algorithm that combines priority-based selection with time-sharing through Round Robin. In this approach, processes are first grouped according to their priority levels, and the CPU is always allocated to the highest-priority group available. Within each priority level, processes are then scheduled using the Round Robin algorithm, where each process receives a fixed time quantum and is cycled through the ready queue.

This hybrid approach aims to balance responsiveness and fairness. Priority scheduling ensures that more important processes are executed first, while Round Robin within each priority level prevents a single process from monopolizing the CPU. This makes the system more suitable for environments where both task importance and interactive responsiveness are required.

2.6.2. Preemptive vs Non-Preemptive Classification

Priority Scheduling with Round Robin is typically classified as a preemptive hybrid scheduling algorithm, depending on the system design. Preemption occurs in two ways:

1. A higher-priority process can interrupt a currently running lower-priority process.
2. Within the same priority level, processes are preempted based on the expiration of their time quantum in the Round Robin cycle.

This dual-level preemption allows the system to remain responsive while still enforcing priority-based execution.

2.6.3. Advantages and Limitations

This hybrid approach improves overall system flexibility by combining the strengths of both algorithms. It ensures that high-priority processes are executed first while still maintaining fairness among processes of the same priority level

through time slicing. It also improves responsiveness in multi-user or time-sharing systems compared to pure Priority Scheduling.

However, the algorithm is more complex to implement due to the need to manage both priority queues and Round Robin cycles. Additionally, if priorities are not well balanced, lower-priority processes may still experience starvation, especially in systems with a continuous flow of high-priority tasks.

Advantages*:*

1. Combines the responsiveness of Priority Scheduling with the fairness of Round Robin
2. Ensures high-priority processes are executed first while maintaining fairness among processes of the same priority level
3. Improves responsiveness in time-sharing and multi-user systems compared to pure Priority Scheduling
4. Prevents starvation within the same priority level through time quantum rotation
5. Suitable for systems requiring both priority-based control and time-sharing

Limitations*:*

1. Low-priority processes may still experience starvation if higher-priority processes continuously arrive
2. More complex to implement due to the need to manage priority queues and Round Robin scheduling
3. Requires careful tuning of both priority levels and time quantum for optimal performance
4. Higher context-switching overhead due to combined preemption mechanisms

2.6.4. Test Case No. 1 (Non-Preemptive Priority + RR, Time Quantum = 2)

The following test case demonstrates the execution of the hybrid Priority Scheduling with Round Robin algorithm in a non-preemptive priority setting. This configuration evaluates how processes are first grouped based on priority levels and then scheduled using Round Robin within each priority group.

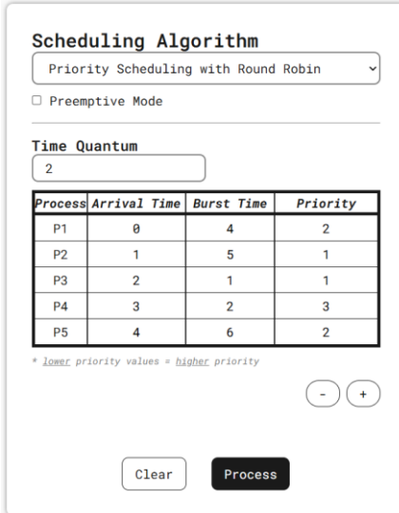


Figure 22. Priority with Round Robin Scheduling – Test Case No. 1 Input Parameter

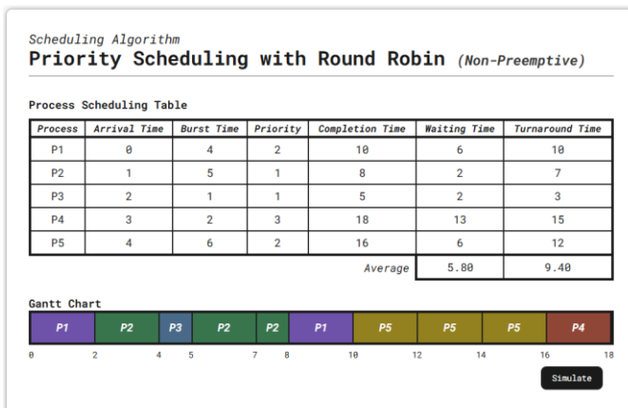


Figure 23. Priority with Round Robin Scheduling – Test Case No. 1 Output

The results show that processes are first organized according to priority levels, and execution is then distributed among processes of the same priority using Round Robin scheduling. This ensures fairness within each priority group while still maintaining overall priority-based execution. However, lower-priority processes experience longer waiting times when higher-priority groups consume CPU time first. The computed performance metrics yield an average waiting time of 5.8 ms and an average turnaround time of 9.4 ms, reflecting balanced fairness within groups but reduced efficiency for lower-priority processes.

2.6.5. Test Case No. 2 (Preemptive Priority + RR, Time Quantum = 3)

This test case evaluates the preemptive version of the Priority + Round Robin hybrid algorithm, where higher-priority processes can interrupt lower-priority execution while maintaining Round Robin scheduling within the same priority level.

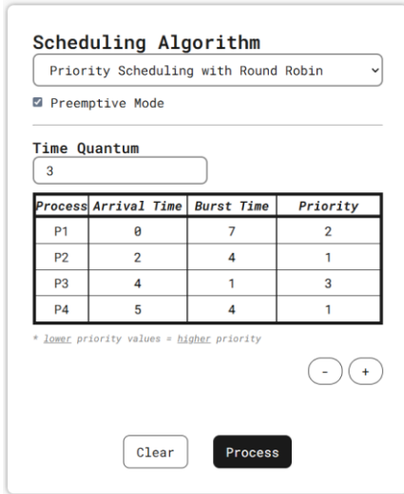


Figure 24. Priority with Round Robin Scheduling – Test Case No. 2 Input Parameter

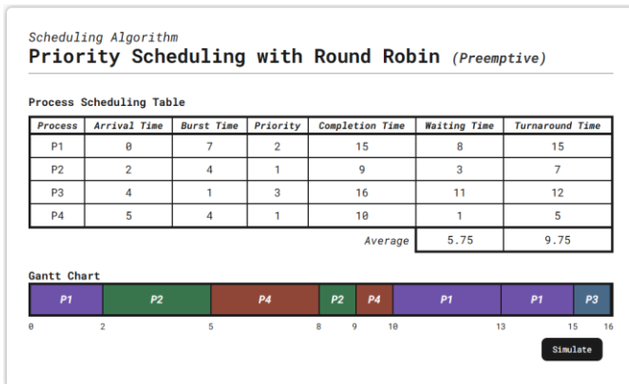


Figure 25. Priority with Round Robin Scheduling – Test Case No. 2 Output

The results demonstrate that higher-priority processes are immediately scheduled upon arrival, even if lower-priority processes are currently executing. Within the same priority level, Round Robin ensures fair CPU distribution. This combination improves responsiveness for critical tasks while maintaining fairness among equal-priority processes. The average waiting time is 5.75 ms, while the average turnaround time is 9.75 ms, indicating improved efficiency compared to the non-preemptive hybrid version.

The hybrid Priority + Round Robin approach effectively balances priority-based execution with time-sharing fairness. The preemptive variant improves responsiveness for high-priority processes, while Round Robin prevents unfair CPU monopolization within the same priority level.

3. Performance Analysis and Discussion

This chapter presents the comparative evaluation of the implemented CPU scheduling algorithms using a standardized workload. The discussion focuses on performance metrics, observed behavior, and the practical implications of each scheduling strategy.

3.1. Comparative Analysis of Algorithms

To systematically evaluate the performance of the implemented CPU scheduling algorithms, a standardized workload was executed across all algorithms. This ensures that all methods are assessed under identical conditions, allowing for fair comparison and meaningful analysis of their respective behaviors and trade-offs.

3.1.1. Standardized Test Workload

The comparative evaluation uses four processes with varying arrival times, burst times, and priority levelss as shown below.

Table 1. Standard Test Workload - Input Parameters

Process	*Arrival Time*	*Burst Time*	*Priority*
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

This workload was designed to reflect realistic multiprogramming scenarios. It includes staggered arrivals, mixed burst durations, and varying priority levels, allowing the evaluation of responsiveness, fairness, and efficiency across all scheduling approaches.

3.1.2. Standardized Test Results

Each scheduling algorithm was executed using the same workload, and performance was measured using Average Waiting Time and Average Turnaround Time.

Table 2. Standardized Test Results

Algorithm	*Average Waiting Time*	*Average Turnaround Time*
FCFS	8.75	15.25
SJF	7.75	14.25
SRTF	6.50	13.00
RR	12.75	19.25
Priority *(Preemptive)*	6.50	13.00

Priority *(Non-Preemptive)*	7.75	14.25
Priority+RR *(Preemptive)*	6.50	13.00
Priority+RR *(Non-Preemptive)*	7.00	13.50

3.1.3. Analysis of Results

The results demonstrate significant variation in performance depending on the scheduling strategy used.

Preemptive Priority Scheduling achieved the best overall performance, with the lowest average waiting and turnaround times. This is attributed to its ability to immediately allocate CPU time to higher-priority processes through preemption.

SRTF also performed efficiently due to its dynamic selection of the shortest remaining process, although its performance is influenced by process arrival timing.

Non-preemptive Priority Scheduling and SJF produced moderate results, benefiting from prioritization and burst-time optimization respectively, but limited by their inability to adapt dynamically to new arrivals.

FCFS served as a baseline algorithm, showing comparatively higher waiting times due to the convoy effect, where shorter processes are delayed by longer ones that arrive earlier.

Round Robin produced the highest waiting and turnaround times in this workload due to frequent context switching introduced by the fixed time quantum, despite its fairness advantages.

The hybrid Priority + Round Robin algorithms demonstrated balanced behavior. The non-preemptive hybrid improved fairness within priority groups, while the preemptive hybrid improved responsiveness by allowing interruption of lower-priority processes. However, both variants still exhibited overhead due to combined scheduling mechanisms.

3.2. Performance Observations

The comparative results reveal several consistent patterns across all scheduling algorithms. Preemptive scheduling algorithms generally outperform non-preemptive variants in terms of responsiveness and average waiting time, particularly in workloads with staggered process arrivals. This is evident in SRTF and Preemptive Priority Scheduling, where dynamic CPU reallocation improves performance for newly arriving or critical processes. A clear trade-off is also observed between fairness and efficiency,

as Round Robin and hybrid Priority + Round Robin scheduling improve fairness by distributing CPU time more evenly among processes, but this results in higher context-switching overhead and increased turnaround time.

The hybrid Priority + Round Robin approach represents a balanced scheduling strategy that combines priority-based execution with time-sharing principles. It improves fairness within priority levels but does not fully eliminate the risk of starvation for lower-priority processes under continuous high-priority load. Additionally, workload characteristics significantly influence performance outcomes, where burst time distribution affects SJF and SRTF, arrival order impacts FCFS, and time quantum selection strongly affects Round Robin performance. Overall, the results confirm that scheduling performance is highly dependent on system goals, and no single algorithm provides optimal performance across all metrics.

3.3. Practical Relevance in Modern Operating Systems

Modern operating systems rarely rely on a single scheduling algorithm. Instead, they employ hybrid scheduling strategies that combine multiple techniques to achieve a balance between fairness, responsiveness, and efficiency across different types of workloads.

For example, Linux Completely Fair Scheduler uses a red-black tree structure to manage processes based on virtual runtime, effectively ensuring fair CPU distribution while maintaining efficient scheduling behavior. Similarly, Microsoft Windows implements multilevel feedback queue scheduling with dynamic priority adjustments and aging mechanisms to improve responsiveness while preventing starvation.

Real-time operating systems typically rely on strict preemptive priority scheduling to guarantee deterministic execution of time-critical tasks. In contrast, modern cloud computing platforms and containerized environments use resource management techniques such as weighted fair scheduling and control group (cgroups)-based allocation to distribute CPU resources proportionally among multiple processes or tenants.

Across these systems, scheduling strategies are designed around core principles such as efficiency, fairness, responsiveness, and overhead management. Although implementation details vary depending on system requirements, these fundamental trade-offs consistently guide scheduler design in both traditional and modern computing environments. CPU scheduling therefore remains a critical component of operating system design, directly influencing overall system performance across embedded systems, personal computers, servers, and large-scale distributed infrastructures.

4. Conclusion

This section presents the overall findings of the study, including the key insights gained from implementing CPU scheduling algorithms, the challenges encountered during development, and possible improvements for enhancing the system in future work.

4.1. Key Insights and Learnings

This project provided a clearer understanding of how CPU scheduling operates within an operating system. It demonstrated that scheduling is not a simple automated process but involves multiple algorithms with distinct trade-offs depending on system requirements.

A key finding is that no single scheduling algorithm is universally optimal. FCFS is simple and fair in execution order but may lead to inefficiency when long processes delay shorter ones. SJF reduces average waiting time under ideal conditions but may cause starvation for longer processes when shorter tasks continuously arrive. These results highlight the trade-off between fairness and efficiency.

Preemptive algorithms such as SRTF and Preemptive Priority Scheduling improve responsiveness by allowing the CPU to switch to more urgent or shorter processes. However, this comes with increased context-switching overhead and implementation complexity. Round Robin ensures fairness through time slicing, but its performance is highly dependent on the time quantum, where poor selection can either increase overhead or reduce responsiveness.

Another important observation is that scheduling performance is highly dependent on workload characteristics such as arrival times and burst durations. This explains why modern operating systems often use hybrid scheduling approaches rather than relying on a single algorithm.

The visualization of process execution through the Gantt chart also helped in better understanding how each algorithm behaves over time.

Key insights obtained from the implementation include:

- Starvation can occur in SJF and Priority Scheduling when shorter or higher-priority processes continuously arrive
- Priority Scheduling often requires aging mechanisms to ensure fairness and prevent indefinite waiting
- SRTF is theoretically optimal in minimizing waiting time but requires knowledge of remaining burst times, which is not always practical

- Combining Priority Scheduling with Round Robin improves fairness within priority levels while maintaining priority-based execution
- Common data structures such as arrays and maps can effectively simulate queues and priority-based scheduling logic in software implementations

4.2. Challenges and Solutions

The implementation of multiple CPU scheduling algorithms introduced several technical and logical challenges in simulation, visualization, and performance evaluation:

Gantt Chart Visualization Implementation

One of the main challenges involved designing a functional and readable Gantt chart visualization. Initial implementations produced static and non-scalable outputs, making it difficult to clearly represent varying workloads and execution timelines. To address this, a responsive layout was implemented using CSS Flexbox, allowing dynamic scaling based on the number of processes and time range. Distinct process colors, structured blocks, and tooltips were also integrated to improve readability and provide detailed process information for each execution segment.

Debugging Preemptive Scheduling Algorithms

Preemptive scheduling algorithms, particularly SRTF, introduced significant complexity due to continuous process monitoring and frequent context switching. Difficulties arose in accurately tracking remaining burst times and selecting the correct process for execution at each time unit. This was resolved by implementing a time-step-based simulation approach, where the system state is updated at every unit of time. Tie-breaking rules based on arrival time and process ID were also added to ensure deterministic behavior, supported by multiple test cases to validate edge conditions.

Accurate Metric Computation in Preemptive Systems

While non-preemptive algorithms allowed straightforward computation of waiting time and turnaround time, preemptive algorithms introduced complexity due to fragmented execution intervals. This made it difficult to correctly account for total waiting periods and completion times. To solve this, a timeline-based tracking system was implemented to record execution segments for each process. Turnaround time was calculated using completion time, while waiting time was derived from total elapsed time minus burst time, ensuring consistent metric computation across all algorithms.

Data Structure Management Across Algorithms

Different scheduling algorithms required different logical structures such as queues, priority sorting, and multi-level scheduling for hybrid implementations.

Managing these varying requirements consistently was initially challenging. This was addressed by primarily using arrays to simulate queues, combined with sorting and filtering for priority-based behavior. For the Priority + Round Robin hybrid algorithm, a map-based structure was introduced to organize processes by priority level while maintaining queue-like execution within each level.

4.3. Future Improvements and Recommendations

Several enhancements can be implemented to further improve the functionality and realism of the system:

- Implementation of real-time Gantt chart animation to visualize process execution dynamically
- Integration of aging mechanisms in Priority Scheduling to prevent starvation
- Inclusion of I/O burst simulation to model realistic process behavior beyond CPU execution
- Development of a comparative execution mode to display all algorithm outputs simultaneously
- Addition of a random workload generator for automated test case creation
- Extension to multi-core CPU scheduling simulation for modern system modeling
- Implementation of Multilevel Feedback Queue scheduling to reflect widely used real-world scheduling strategies
- Improvement of input validation and error handling for robustness

5. References

Catlin, B., Hanrahan, J. E., Russinovich, M. E., Solomon, D. A., & Ionescu, A. (2017). *Windows Internals*. Microsoft.

Liu, J. W. S. (2006). *Real-Time systems*. Pearson Education.

Love, R. (2018). *Linux Kernel Development*. Pearson Education India.

Silberschatz, A., Galvin, P., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley, Cop.

Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson Education Limited.

Tanenbaum, A. S., & Bos, H. (2022). *Modern Operating Systems* (5th ed.). Pearson Education.

6. Appendices

6.1. Repository Link

https://github.com/rn-glngn/CPU-Scheduling-Algorithm

6.2. Executable File/ Link

https://rn-glngn.github.io/CPU-Scheduling-Algorithm/

6.3. Source Code

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <link rel="stylesheet"
href="globals.css">
  <link rel="stylesheet"
href="style.css">
  <title>CPU Scheduling Algorithm</title>
  <link type="image/png" sizes="96x96"
rel="icon"
href="https://img.icons8.com/sf-
black/64/electronics.png">
  <script type="module"
src="./js/main.js" defer></script>
</head>

<body>

  <!-- ==============================
       HEADER SECTION
       ============================== -->
  <header class="title-section">
    <img width="64" height="64"
src="https://img.icons8.com/sf-
black/64/electronics.png"
alt="electronics" />
    <h1>CPU Scheduling Algorithm</h1>
  </header>

  <main>

    <!-- ==============================
         USER INPUT SECTION
         ============================== -
->
    <div class="user-input card">

      <!-- Scheduling algorithm selector
-->
      <h2>Scheduling Algorithm</h2>
      <select name="sched-algo"
id="sched-algo">

        <!-- Different CPU scheduling
algorithms -->
        <option value="fcfs">First-Come,
First-Served (FCFS)</option>
        <option value="sjf">Shortest Job
First (SJF)</option>
        <option value="srtf">Shortest
Remaining Time First (SRTF)</option>
        <option value="rr">Round Robin
(RR)</option>
        <option value="priority">Priority
Scheduling</option>
        <option value="priority-
rr">Priority Scheduling with Round
Robin</option>

      </select>

      <!-- Preemptive toggle (shown only
for applicable algorithms) -->
      <div id="preemptive-container"
style="display: none;">
        <label>
          <input type="checkbox"
id="preemptive-toggle">
          Preemptive Mode
        </label>
      </div>

      <hr>

      <!-- Time Quantum input (only
visible for RR-based algorithms) -->
      <div id="time-quantum-container"
style="display: none;">
        <h3>Time Quantum</h3>
        <input type="number" id="time-
quantum" class="sched-algo" min="1"
placeholder="2">
      </div>

      <!-- ==============================
           PROCESS INPUT TABLE
           Users define process details
here:
           - Arrival Time
           - Burst Time
           - Priority (optional)
           ========================== -->
      <table id="input-table"
class="sched-table">
        <thead>
          <tr>
            <th>Process</th>
            <th>Arrival Time</th>
            <th>Burst Time</th>

            <!-- Priority column hidden
for non-priority algorithms -->
            <th class="prio-col hide-
col">Priority</th>
          </tr>
        </thead>

        <tbody id="table-body">

          <!-- Default process rows -->
          <tr>
```

```
            <td>P1</td>
            <td><input type="number"
placeholder="0" min="0"></td>
            <td><input type="number"
placeholder="1" min="1"></td>
            <td class="prio-col hide-
col"><input type="number" placeholder="0"
min="0"></td>
          </tr>

          <tr>
            <td>P2</td>
            <td><input type="number"
placeholder="0" min="0"></td>
            <td><input type="number"
placeholder="1" min="1"></td>
            <td class="prio-col hide-
col"><input type="number" placeholder="0"
min="0"></td>
          </tr>

          <tr>
            <td>P3</td>
            <td><input type="number"
placeholder="0" min="0"></td>
            <td><input type="number"
placeholder="1" min="1"></td>
            <td class="prio-col hide-
col"><input type="number" placeholder="0"
min="0"></td>
          </tr>

        </tbody>
      </table>

      <!-- Priority rule explanation -->
      <p id="prio-caption">
        * <u>lower</u> priority values =
<u>higher</u> priority
      </p>

      <!-- Row control buttons -->
      <div class="action-btns">
        <button id="remove-row-btn">-
</button>
        <button id="add-row-
btn">+</button>
      </div>

      <!-- Main action buttons -->
      <div class="btn-grp">
        <button id="clear-
btn">Clear</button>
        <button id="process-
btn">Process</button>
      </div>

    </div>

    <!-- ============================
         RESULTS SECTION
         Displays computed scheduling
output:
         - Process table
         - Metrics (avg waiting &
turnaround)
         - Gantt chart visualization
         ============================ -->
    <div class="results card">

      <!-- Selected algorithm display -->
      <div class="subtitle-section">
        <h3>Scheduling Algorithm</h3>
        <h2 id="algo-choice">
          First-Come First Serve (FCFS)
          <span id="preemptive-
choice"></span>
        </h2>
      </div>

      <!-- ==========================
           OUTPUT TABLE
           Shows computed results per
process:
           - Completion Time
           - Waiting Time
           - Turnaround Time
           ========================== -->
      <div class="table-section">
        <h4>Process Scheduling Table</h4>

        <table id="result-table"
class="sched-table">
          <thead>
            <tr>
              <th>Process</th>
              <th>Arrival Time</th>
              <th>Burst Time</th>
              <th>Priority</th>
              <th>Completion Time</th>
              <th>Waiting Time</th>
              <th>Turnaround Time</th>
            </tr>
          </thead>

          <tbody>

            <!-- Result rows (filled
dynamically via JS) -->
            <tr>
              <td>P1</td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
            </tr>

            <tr>
              <td>P2</td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
            </tr>

            <tr>
              <td>P3</td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
              <td></td>
            </tr>

            <!-- Average metrics row -->
            <tr id="metrics">
              <td
colspan="5">Average</td>
              <td colspan="1"></td>
              <td colspan="1"></td>
            </tr>

          </tbody>
        </table>
      </div>

      <br>

      <!-- ==========================
```

```
          GANTT CHART SECTION
          ========================= -->
      <div class="gantt-chart-section">
        <h4>Gantt Chart</h4>

        <div id="gantt-chart-container">

          <!-- Bars representing process
execution -->
          <div id="gantt-bar-
container"></div>

          <!-- Timeline labels -->
          <div id="gantt-timeline"></div>
```

```
        </div>

        <!-- Simulation trigger -->
        <div class="sim-grp">
          <button id="sim-
btn">Simulate</button>
        </div>
      </div>

    </div>

  </main>
</body>

</html>
```

globals.css

```
@import
url('https://fonts.googleapis.com/css2?fa
mily=Roboto+Mono:ital,wght@0,100..700;1,1
00..700&display=swap');

/* ==================================
   CSS RESET
   ================================== */
*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html,
body {
  height: 100%;
  scroll-behavior: smooth;
}

body {
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
  padding: 40px 50px;
  touch-action: manipulation;
}

ul,
ol {
  list-style: none;
}

a {
  text-decoration: none;
  color: inherit;
}

img,
picture,
video,
canvas,
svg {
  display: block;
  max-width: 100%;
}

input,
button,
textarea,
select {
  font: inherit;
}

/* ==================================
   ROOT VARIABLES
   ================================== */
:root {
  /* Colors */
  --color-bg: #fafafa;
  --color-primary: #1a1a1a;

  /* Typography */
  --font-sans: 'Roboto Mono', sans-serif;

  /* Spacing */
  --spacing-xs: 0.25rem;
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 2rem;
  --spacing-xl: 4rem;

  /* Borders */
  --border-xs: 1px;
  --border-sm: 2px;
  --border-md: 4px;
  --border-lg: 8px;
  --border-xl: 16px;
}

/* ==================================
   BASE STYLES
   ================================== */
body {
  font-family: var(--font-sans);
  background-color: var(--color-bg);
  color: var(--color-primary);
}

h1,
h2,
h3,
h4,
h5,
h6 {
  font-weight: 600;
  line-height: 1.2;
}

p {
  margin-bottom: var(--spacing-md);
}

/* ==================================
   BUTTONS
   ================================== */
button {
  cursor: pointer;
  border: none;
  border-radius: var(--border-lg);
  padding: var(--spacing-sm) var(--
spacing-md);
  background-color: var(--color-primary);
  color: white;
  transition: 0.2s ease;
}
```

```
button:hover {
  opacity: 0.9;
  transform: scale(1.1);
}

/* ==================================
   UTILITIES
   ================================== */
.hidden,
.hide-col {
  display: none;
}

/* ==================================
   FORM ELEMENTS
   ================================== */

/* Dropdown */
select {
  border: var(--border-sm) solid #7b7b7b;
  border-radius: var(--border-lg);
  padding: var(--spacing-sm) var(--
spacing-md);
  background-color: white;
  color: var(--color-primary);
}
```

```
option:hover {
  background-color: #7b7b7b;
}

/* Number Inputs */
input[type="number"] {
  width: 100%;
  padding: var(--spacing-xs);
  text-align: center;
  border: none;
}

/* Focused input styling */
input[type="number"]:active,
#input-table input[type="number"]:focus {
  outline: none;
  background-color: var(--color-primary);
  opacity: 0.75;
  color: var(--color-bg);
  font-weight: bold;
}

/* Remove browser spinner buttons */
input::-webkit-inner-spin-button,
input::-webkit-outer-spin-button {
  -webkit-appearance: none;
  margin: 0;
```

styles.css

```
@import url('globals.css');

/* ==================================
   HEADER SECTION
   ================================== */
header {
  display: flex;
  align-items: center;
  gap: var(--spacing-sm);
}

/* ==================================
   MAIN LAYOUT STRUCTURE
   ================================== */
main {
  padding: var(--spacing-md) var(--
spacing-lg);
  display: grid;
  grid-template-columns: 1fr 1.6fr;
  gap: var(--spacing-lg);
  align-items: start;
}

/* ==================================
   CARD COMPONENT
   ================================== */
.card {
  background-color: white;
  padding: var(--spacing-lg);
  border-radius: var(--border-lg);
  box-shadow: 0 0px 6px rgba(0, 0, 0,
0.5);

  min-height: 500px;
  height: auto;
  overflow: auto;
}

/* ==================================
   INPUT CONTROLS
   ================================== */

/* Dropdown selector for scheduling
algorithm */
select {
```

```
  margin-bottom: var(--spacing-sm);
  width: 100%;
  max-width: 100%;
  box-sizing: border-box;
}

/* Divider line spacing */
.user-input hr {
  margin-top: var(--spacing-md);
  margin-bottom: var(--spacing-md);
}

/* Time quantum input (Round Robin based
algorithms) */
#time-quantum {
  width: 50%;
  text-align: left;
  border: var(--border-sm) solid #7b7b7b;
  border-radius: var(--border-lg);
  padding: var(--spacing-xs) var(--
spacing-sm);
  padding-left: var(--spacing-md);
  background-color: white;
  color: var(--color-primary);
}

/* Process input table spacing */
#input-table {
  margin-bottom: var(--spacing-sm);
}

/* Priority explanation text (hidden by
default) */
#prio-caption {
  font-size: 0.75rem;
  color: #7b7b7b;
  font-style: italic;
  display: none;
}

/* ==================================
   ROW CONTROL BUTTONS (- / +)
   ================================== */
.action-btns {
  display: flex;
```

```
  justify-content: flex-end;
  gap: var(--spacing-sm);
  margin-bottom: var(--spacing-md);
}

.action-btns button {
  padding: var(--spacing-xs) var(--
spacing-md);
  border-radius: var(--border-xl);
  border: var(--border-xs) solid var(--
color-primary);
  background-color: white;
  color: var(--color-primary);
}

/* Hover effect for row controls */
.action-btns button:hover {
  background-color: var(--color-primary);
  color: white;
}

/* ==================================
   MAIN ACTION BUTTON GROUP (Clear /
Process)
   ================================== */
.btn-grp {
  display: flex;
  gap: var(--spacing-lg);
  justify-content: center;
  margin-top: var(--spacing-xl);
}

/* Clear button styling */
.btn-grp #clear-btn {
  border: var(--border-xs) solid var(--
color-primary);
  background-color: white;
  color: var(--color-primary);
}

/* ==================================
   SHARED TABLE STYLING
   ================================== */
.sched-table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 10px;
  border: var(--border-md) solid var(--
color-primary);
  text-align: center;
  overflow: auto;
}

/* Table header styling */
.sched-table thead {
  border-bottom: var(--color-primary)
solid;
  border-width: var(--border-md);
  font-weight: bold;
  font-style: italic;
}

/* Cell borders */
.sched-table td,
.sched-table th {
  border: var(--border-xs) solid var(--
color-primary);
  border-right: var(--border-sm) solid;
}

/* ==================================
   RESULTS HEADER SECTION
   ================================== */
.subtitle-section {
  margin-bottom: var(--spacing-lg);
  border-bottom: var(--border-xs) solid
#7b7b7b;
  padding-bottom: var(--spacing-sm);
}

.subtitle-section h3 {
  font-weight: normal;
  font-style: italic;
}

.subtitle-section h2 {
  font-size: 1.75rem;
  font-weight: bolder;
}

h4 {
  font-weight: bold;
}

/* Sub-label for preemptive mode
indicator */
#preemptive-choice {
  font-size: 1.25rem;
  color: #393939;
  font-style: italic;
}

/* ==================================
   RESULT TABLE (OUTPUT TABLE)
   ================================== */
#result-table {
  border: none;
}

#result-table td,
#result-table th {
  border: var(--border-xs) solid var(--
color-primary);
  border-right: var(--border-sm) solid;
  border-bottom: var(--border-sm) solid;
  padding: var(--spacing-xs) 0;
}

#result-table th {
  padding: 0 var(--spacing-xs);
  font-size: 0.9rem;
}

/* Strong borders on first/last columns
*/
#result-table td:first-child {
  border-left: var(--border-md) solid
var(--color-primary);
}

#result-table th:last-child,
#result-table td:last-child {
  border-right: var(--border-md) solid
var(--color-primary);
}

/* Header border emphasis */
#result-table thead {
  border: var(--border-md) solid var(--
color-primary);
}

/* Separator before metrics row */
#result-table tbody tr:last-child {
  border-top: var(--border-md) solid
var(--color-primary);
}

/* Metrics row styling */
#result-table tr:last-child td:first-
child {
  border: none;
}
```

```
#result-table tr:last-child td:last-child
{
  border-right: var(--border-md) solid
var(--color-primary);
  border-bottom: var(--border-md) solid
var(--color-primary);
}

#result-table tr:last-child td:nth-last-
child(2) {
  border-left: var(--border-md) solid
var(--color-primary);
  border-bottom: var(--border-md) solid
var(--color-primary);
}

#metrics td:first-child {
  font-style: italic;
  text-align: right;
  padding-right: var(--spacing-sm);
}

/* ==================================
   GANTT CHART SECTION
   ================================== */

/* Enables horizontal scrolling if chart
overflows */
.table-section,
#gantt-chart-container {
  overflow-x: auto;
}

/* Main bar container */
#gantt-bar-container {
  display: flex;
  width: 100%;
  height: 50px;
  border: var(--border-md) solid var(--
color-primary);
}

/* Individual process blocks in Gantt
chart */
.gantt-block {
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: bold;
  font-style: italic;
  border-right: var(--border-sm) solid
var(--color-primary);
  color: var(--color-bg);
  flex: 1 1 0%;
  transition: all 0.3s ease;
}

/* Timeline labels under Gantt chart */
#gantt-timeline {
  display: flex;
  width: 100%;
  font-size: 0.75rem;
  margin-top: 5px;
  position: relative;
}

/* Simulation button wrapper */
.sim-grp {
  font-size: 0.75rem;
  padding: var(--spacing-sm) var(--
spacing-md);
  display: flex;
  justify-content: flex-end;
}

/* ==================================
   RESPONSIVE DESIGN
   ================================== */

/* Tablet screens */
@media (max-width: 1024px) {
  main {
    grid-template-columns: 1fr;
    gap: var(--spacing-md);
  }

  .card {
    min-height: auto;
  }

  #time-quantum {
    width: 100%;
  }

  .action-btns {
    justify-content: center;
  }
}

/* Mobile screens */
@media (max-width: 768px) {
  body {
    padding: 20px;
  }

  header {
    flex-direction: column;
    text-align: center;
  }

  main {
    padding: var(--spacing-sm);
  }

  .card {
    padding: var(--spacing-md);
  }

  .subtitle-section h2 {
    font-size: 1.25rem;
  }

  .sched-table {
    font-size: 0.85rem;
  }

  button {
    width: 100%;
  }

  .btn-grp {
    flex-direction: column;
    gap: var(--spacing-sm);
    margin-top: var(--spacing-md);
  }

  .action-btns {
    flex-direction: row;
    justify-content: space-between;
  }
}

/* Small mobile screens */
@media (max-width: 480px) {
  .subtitle-section h2 {
    font-size: 1rem;
  }

  .sched-table {
    font-size: 0.75rem;
  }

  h4 {
    font-size: 0.9rem;
```

```
  }

  #gantt-bar-container {
    height: 40px;
  }
}
```

main.js

```
import { getInput, getTimeQuantum } from
"./inputs/getInput.js";
import { validateInput } from
"./inputs/validateInput.js";

import { fcfs } from
"./algorithms/fcfs.js";
import { sjf } from
"./algorithms/sjf.js";
import { srtf } from
"./algorithms/srtf.js";
import { rr } from "./algorithms/rr.js";
import {
  priorityPreemptive,
  priorityNonPreemptive,
} from "./algorithms/priority.js";
import {
  priorityRRPreemptive,
  priorityRRNonPreemptive,
} from "./algorithms/priorityRR.js";

import { renderTable } from
"./output/renderTable.js";
import { renderGantt } from
"./output/renderGantt.js";
import { simulateGantt } from
"./output/simulateGantt.js";

export function toggleInputField(algo) {
  const selected = algoSelect.value;
  const priorityCells =
document.querySelectorAll(".prio-col");
  const show = algo.includes("priority");

  if (selected === "rr" || selected ===
"priority-rr") {
    tqContainer.style.display = "block";
  } else {
    tqContainer.style.display = "none";
  }

  priorityCells.forEach((cell) => {
    if (show) {
      cell.classList.remove("hide-col");
    } else {
      cell.classList.add("hide-col");
    }
  });
}

export function
togglePreemptiveInput(algo) {
  const container =
document.getElementById("preemptive-
container");
  const note =
document.getElementById("prio-caption");

  const show = algo.includes("priority");

  container.style.display = show ?
"block" : "none";
  note.style.display = show ? "block" :
"none";
}

// Show/hide input controls based on
selected algorithm
const algoSelect =
document.getElementById("sched-algo");
const tqContainer =
document.getElementById("time-quantum-
container");
const prioColElems =
document.querySelectorAll(".prio-col");
let currentGantt = [];

algoSelect.addEventListener("change", ()
=> {
  const selected = algoSelect.value;

  toggleInputField(selected);
  togglePreemptiveInput(selected);
});

// Clear input table and reset to default
state
const clearBtn =
document.getElementById("clear-btn");
const tableBody =
document.getElementById("table-body");
clearBtn.addEventListener("click", () =>
{
  const tq =
document.getElementById("time-quantum-
container");
  document.getElementById("time-
quantum").value = "";

  tableBody.innerHTML = "";

  for (let i = 1; i <= 2; i++) {
    const row =
document.createElement("tr");

    row.innerHTML = `
      <td>P${i}</td>
      <td><input type="number"
placeholder="0" min="0"></td>
      <td><input type="number"
placeholder="1" min="1"></td>
      <td class="prio-col hide-
col"><input type="number" placeholder="0"
min="0"></td>
    `;

    tableBody.appendChild(row);
  }

  toggleInputField(algoSelect.value);

  clearOutputs();
});

// Main entry point - handles button
click to process input data
document.getElementById("process-
btn").addEventListener("click", () => {
  const processes = getInput();

  if (!validateInput(processes)) return;

  const algo =
document.getElementById("sched-
algo").value;
  const isPreemptive =
document.getElementById("preemptive-
toggle").checked;

  const baseText =
algoSelect.options[algoSelect.selectedInd
ex].text;
```

```
  document.getElementById("algo-
choice").childNodes[0].textContent =
    baseText + " ";

  const preemptiveSpan =
document.getElementById("preemptive-
choice");

  if (algo.includes("priority")) {
    preemptiveSpan.textContent =
isPreemptive
      ? "(Preemptive)"
      : "(Non-Preemptive)";
  } else {
    preemptiveSpan.textContent = "";
  }

  let results;

  if (algo === "fcfs") {
    results = fcfs(processes);
  } else if (algo === "sjf") {
    results = sjf(processes);
  } else if (algo === "srtf") {
    results = srtf(processes);
  } else if (algo === "rr") {
    const timeQuantum = getTimeQuantum();
    if (timeQuantum === null) return;
    results = rr(processes, timeQuantum);
  } else if (algo === "priority") {
    results = isPreemptive
      ? priorityPreemptive(processes)
      : priorityNonPreemptive(processes);
  } else if (algo === "priority-rr") {
    const timeQuantum = getTimeQuantum();
    if (timeQuantum === null) return;
    results = isPreemptive
      ? priorityRRPreemptive(processes,
timeQuantum)
      :
priorityRRNonPreemptive(processes,
timeQuantum);
  }

  renderTable(results);
  renderGantt(results.ganttChart);
  currentGantt = results.ganttChart;
});

document.getElementById("sim-
btn").addEventListener("click", () => {
  if (!currentGantt.length) return;
  simulateGantt(currentGantt);
});
```

fcfs.js

```
// ==================================
// FIRST-COME FIRST-SERVED (FCFS)
SCHEDULING ALGORITHMS
// ==================================
//
// Processes are executed in the order
they arrive in the ready queue.
//
// ==================================

export function fcfs(processes) {
  // -----------------------------------
  // Sort processes based on arrival time
  // -----------------------------------
  // FCFS executes processes in the exact
order they arrive.
  // Earlier arrival time = higher
execution priority.
  //
  // We create a copy using
[...processes] to avoid modifying the
original array.
  const sorted = [...processes].sort((a,
b) => a.arrival - b.arrival);

  // Tracks the current CPU time during
simulation
  let currentTime = 0;

  // Stores execution segments for Gantt
Chart visualization
  const ganttChart = [];

  // Stores computed scheduling data for
each process
  const table = [];

  // -----------------------------------
  // Process each task sequentially
  // -----------------------------------
  sorted.forEach((p) => {
    // CPU starts either:
    // - when current process arrives
    // - OR when previous process
finishes
    //
    // This also handles CPU idle time.
    const start = Math.max(currentTime,
p.arrival);

    // Completion time of the process
    const end = start + p.burst;

    // Turnaround Time = Completion Time
- Arrival Time
    const turnaround = end - p.arrival;

    // Waiting Time = Turnaround Time -
Burst Time
    const waiting = turnaround - p.burst;

    // ----------------------------------
    // Store Gantt Chart information
    // ----------------------------------
    ganttChart.push({
      id: p.id, // Process ID
      start, // Start execution time
      end, // End execution time
      duration: end - start, // Total
runtime
    });

    // ----------------------------------
    // Store scheduling metrics in result
table
    // ----------------------------------
    table.push({
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      completion: end,
      turnaround,
      waiting,
    });

    // Update CPU time for next process
    currentTime = end;
  });

  // -----------------------------------
  // Compute average waiting time and
average turnaround time
```

```
  // ------------------------------------
  const avgWaiting =
    table.reduce((sum, p) => sum +
p.waiting, 0) / table.length;

  const avgTurnaround =
    table.reduce((sum, p) => sum +
p.turnaround, 0) / table.length;

  // ------------------------------------
  // FINAL OUTPUT
  // ------------------------------------
  return {
    // Used for visual Gantt Chart
rendering
    ganttChart,

    // Contains all computed process
metrics
    table,

    // Overall scheduling performance
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}
```

sjf.js

```
// ======================================
// SHORTEST JOB FIRST (SJF) SCHEDULING
ALGORITHM
// ======================================
//
// The CPU always selects the process
with the SMALLEST burst time
// among all currently available
processes.
//
// Once a process starts executing, it
runs until completion.
//
// ======================================

export function sjf(processes) {
  // Total number of processes
  const n = processes.length;

  // Stores completed processes
  const completed = [];

  // Stores execution timeline for
visualization
  const ganttChart = [];

  // Stores process scheduling metrics
  const table = [];

  // Current CPU time
  let currentTime = 0;

  // ------------------------------------
  // Clone process list and add
completion tracker
  // ------------------------------------
  let remaining = [...processes].map((p)
=> ({
    ...p,

    // Indicates if process already
finished
    done: false,
  }));

  // ======================================
  // MAIN SCHEDULING LOOP
  // Continues until all processes are
completed
  // ======================================
  while (completed.length < n) {
    // ----------------------------------
    // Get all available processes
    // Conditions:
    // • Process already arrived
    // • Process not yet completed
    // ----------------------------------
    const available = remaining.filter(
      (p) => !p.done && p.arrival <=
currentTime,
    );

    // ----------------------------------
    // CPU IDLE CONDITION
    // ----------------------------------
    // If no process has arrived yet,
    // advance CPU time by 1 unit.
    // ----------------------------------
    if (available.length === 0) {
      currentTime++;

      continue;
    }

    // ----------------------------------
    // Select process with shortest burst
time
    // ----------------------------------
    available.sort((a, b) => a.burst -
b.burst);

    // Chosen process
    const current = available[0];

    // ----------------------------------
    // Calculate execution times
    // ----------------------------------
    const start = currentTime;

    const end = start + current.burst;

    // Completion Time
    const completion = end;

    // Turnaround Time
    // = Completion - Arrival
    const turnaround = completion -
current.arrival;

    // Waiting Time
    // = Turnaround - Burst
    const waiting = turnaround -
current.burst;

    // ----------------------------------
    // Store execution segment in Gantt
Chart
    // ----------------------------------
    ganttChart.push({
      id: current.id,
      start,
      end,
      duration: end - start,
    });

    // ----------------------------------
    // Store process metrics
    // ----------------------------------
    table.push({
      id: current.id,
```

```
      arrival: current.arrival,
      burst: current.burst,

      // Included for compatibility with
other algorithms
      priority: current.priority,

      completion,
      waiting,
      turnaround,
    });

    // ----------------------------------
    // Update CPU state
    // ----------------------------------

    // Move CPU time forward
    currentTime = end;

    // Mark process as completed
    current.done = true;

    // Add to completed list
    completed.push(current);
  }

  // -------------------------------------
  // Compute Average Waiting Time
  // -------------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  // -------------------------------------
  // Compute Average Turnaround Time
  // -------------------------------------
  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;

  // -------------------------------------
  // FINAL OUTPUT
  // -------------------------------------
  return {
    // Used for Gantt Chart visualization
    ganttChart,

    // Scheduling metrics per process
    table,

    // Overall algorithm performance
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}
```

srtf.js

```
// ======================================
// SHORTEST REMAINING TIME FIRST (SRTF)
SCHEDULING ALGORITHM
// ======================================
//
// Shortest Remaining Time First (SRTF)
is the PREEMPTIVE version
// of the Shortest Job First (SJF)
scheduling algorithm.
//
// The CPU always executes the process
with the SMALLEST remaining
// burst time among all available
processes.
//
// If a newly arrived process has a
shorter remaining time than the
// currently running process, the CPU
immediately interrupts
// (preempts) the current process.
//
// ======================================

export function srtf(processes) {
  // Total number of processes
  const n = processes.length;

  // Current CPU time
  let time = 0;

  // Number of completed processes
  let completed = 0;

  // -------------------------------------
  // Clone processes and track remaining
burst time
  // -------------------------------------
  const remaining = processes.map((p) =>
({
    ...p,

    // Remaining burst time
    remaining: p.burst,

    // Completion tracker
    completed: false,

  }));

  // Stores execution timeline
  const ganttChart = [];

  // Stores scheduling metrics
  const tableMap = new Map();

  // -------------------------------------
  // Initialize process table
  // -------------------------------------
  processes.forEach((p) => {
    tableMap.set(p.id, {
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      completion: 0,
      waiting: 0,
      turnaround: 0,
    });
  });

  // Tracks currently running process
  let lastProcess = null;

  // Start time of current Gantt segment
  let segmentStart = 0;

  // ====================================
  // MAIN SCHEDULING LOOP
  // ====================================
  while (completed < n) {
    // ----------------------------------
    // Get all available processes
    // Conditions:
    // • Already arrived
    // • Not completed
    // • Still has remaining burst time
    // ----------------------------------
    const available = remaining.filter(
      (p) => p.arrival <= time &&
!p.completed && p.remaining > 0,
    );

    // ----------------------------------
    // CPU IDLE CONDITION
```

```
    // ----------------------------------
    if (available.length === 0) {
      // --------------------------------
      // Close currently running segment
before idling
      // --------------------------------
      if (lastProcess !== null) {
        ganttChart.push({
          id: lastProcess,
          start: segmentStart,
          end: time,
          duration: time - segmentStart,
        });

        lastProcess = null;
      }

      // --------------------------------
      // Find next arriving process
      // --------------------------------
      const nextArrival = Math.min(
        ...remaining
          .filter((p) => !p.completed &&
p.arrival > time)
          .map((p) => p.arrival),
      );

      // --------------------------------
      // Merge consecutive IDLE segments
      // --------------------------------
      const last =
ganttChart[ganttChart.length - 1];

      if (last && last.id === "IDLE") {
        last.end = nextArrival;

        last.duration = last.end -
last.start;
      } else {
        ganttChart.push({
          id: "IDLE",
          start: time,
          end: nextArrival,
          duration: nextArrival - time,
        });
      }

      // Jump CPU time forward
      time = nextArrival;

      continue;
    }

    // ----------------------------------
    // Select process with SHORTEST
remaining burst time
    // ----------------------------------
    available.sort((a, b) => {
      // Tie breaker:
      // Earlier arrival executes first
      if (a.remaining === b.remaining) {
        return a.arrival - b.arrival;
      }

      return a.remaining - b.remaining;
    });

    // Selected process
    const current = available[0];

    // ----------------------------------
    // PROCESS SWITCH / PREEMPTION
    // ----------------------------------
    // If current process changes,
    // save previous execution segment
    // ----------------------------------
    if (lastProcess !== current.id) {
      // Save previous segment
      if (lastProcess !== null) {
        ganttChart.push({
          id: lastProcess,
          start: segmentStart,
          end: time,
          duration: time - segmentStart,
        });
      }

      // Start new execution segment
      lastProcess = current.id;

      segmentStart = time;
    }

    // ----------------------------------
    // Execute process for 1 time unit
    // ----------------------------------
    current.remaining--;

    // Advance CPU time
    time++;

    // ----------------------------------
    // PROCESS COMPLETION CHECK
    // ----------------------------------
    if (current.remaining === 0) {
      current.completed = true;

      completed++;

      const p = tableMap.get(current.id);

      // Completion Time
      p.completion = time;

      // Turnaround Time
      // = Completion - Arrival
      p.turnaround = p.completion -
p.arrival;

      // Waiting Time
      // = Turnaround - Burst
      p.waiting = p.turnaround - p.burst;
    }
  }

  // ------------------------------------
  // Save final Gantt segment
  // ------------------------------------
  if (lastProcess !== null &&
segmentStart !== time) {
    ganttChart.push({
      id: lastProcess,
      start: segmentStart,
      end: time,
      duration: time - segmentStart,
    });
  }

  // Convert Map → Array
  const table =
Array.from(tableMap.values());

  // ------------------------------------
  // Compute Average Waiting Time
  // ------------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  // ------------------------------------
  // Compute Average Turnaround Time
  // ------------------------------------
  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;
```

```
  // ------------------------------------
  // FINAL OUTPUT
  // ------------------------------------
  return {
    // Gantt Chart visualization data
    ganttChart,

    // Scheduling metrics
    table,

    // Overall scheduling performance
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}
```

rr.js

```
// ======================================
// ROUND ROBIN (RR) SCHEDULING ALGORITHM
// ======================================
//
// If the process is not finished after
its quantum expires,
// it is placed back at the end of the
ready queue.
//
// This algorithm is commonly used in
time-sharing systems
// because it provides fair CPU
allocation.
//
// ======================================

export function rr(processes,
timeQuantum) {
  // Total number of processes
  const n = processes.length;

  // Current CPU time
  let time = 0;

  // Number of completed processes
  let completed = 0;

  // ------------------------------------
  // Clone processes and add remaining
burst tracker
  // ------------------------------------
  const remaining = processes.map((p) =>
({
    ...p,

    // Remaining burst time
    remaining: p.burst,

    // Completion tracker
    completed: false,
  }));

  // Stores execution segments for Gantt
Chart
  const ganttChart = [];

  // Stores computed scheduling metrics
  const tableMap = new Map();

  // ------------------------------------
  // Initialize scheduling table
  // ------------------------------------
  processes.forEach((p) => {
    tableMap.set(p.id, {
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      completion: 0,
      waiting: 0,
      turnaround: 0,
    });
  });

  // ------------------------------------
  // Ready Queue
  // Processes waiting for CPU execution
  // ------------------------------------
  const queue = [];

  // Tracks current running process in
Gantt Chart
  let lastProcess = null;

  // Start time of current Gantt segment
  let segmentStart = 0;

  // ------------------------------------
  // Sort processes based on arrival time
  // ------------------------------------
  remaining.sort((a, b) => a.arrival -
b.arrival);

  // Arrival pointer
  // Tracks next process to arrive
  let i = 0;

  // ====================================
  // MAIN SCHEDULING LOOP
  // ====================================
  while (completed < n) {
    // ----------------------------------
    // Add newly arrived processes into
Ready Queue
    // ----------------------------------
    while (i < n && remaining[i].arrival
<= time) {
      queue.push(remaining[i]);

      i++;
    }

    // ----------------------------------
    // CPU IDLE CONDITION
    // ----------------------------------
    // If no process exists in the ready
queue,
    // CPU remains idle until next
process arrives.
    // ----------------------------------
    if (queue.length === 0) {
      // Close currently running Gantt
segment
      if (lastProcess !== null) {
        ganttChart.push({
          id: lastProcess,
          start: segmentStart,
          end: time,
          duration: time - segmentStart,
        });

        lastProcess = null;
      }

      // No remaining arriving processes
      if (i >= n) break;

      // Next process arrival time
      const nextArrival =
remaining[i].arrival;
```

```
      // --------------------------------
      // Merge consecutive IDLE segments
      // --------------------------------
      const last =
ganttChart[ganttChart.length - 1];

      if (last && last.id === "IDLE") {
        last.end = nextArrival;
        last.duration = last.end -
last.start;
      } else {
        ganttChart.push({
          id: "IDLE",
          start: time,
          end: nextArrival,
          duration: nextArrival - time,
        });
      }

      // Move CPU time forward
      time = nextArrival;

      continue;
    }

    // ----------------------------------
    // Get next process from Ready Queue
(FIFO)
    // ----------------------------------
    const current = queue.shift();

    // ----------------------------------
    // Start new Gantt segment if process
changes
    // ----------------------------------
    if (lastProcess !== current.id) {
      // Save previous segment
      if (lastProcess !== null) {
        ganttChart.push({
          id: lastProcess,
          start: segmentStart,
          end: time,
          duration: time - segmentStart,
        });
      }

      // Start new process segment
      lastProcess = current.id;

      segmentStart = time;
    }

    // ----------------------------------
    // Determine execution duration
    // ----------------------------------
    // Execute either:
    // - remaining burst time
    // - OR time quantum
    //
    // whichever is smaller
    // ----------------------------------
    const execTime =
Math.min(timeQuantum, current.remaining);

    // Reduce remaining burst time
    current.remaining -= execTime;

    // Advance CPU time
    time += execTime;

    // ----------------------------------
    // Add processes arriving during
execution
    // ----------------------------------
    while (i < n && remaining[i].arrival
<= time) {
      queue.push(remaining[i]);
      i++;
    }

    // ----------------------------------
    // PROCESS REQUEUE / COMPLETION
    // ----------------------------------

    // If process still needs CPU time,
    // place it back at end of queue
    if (current.remaining > 0) {
      queue.push(current);
    } else {
      // Process completed
      current.completed = true;

      completed++;

      const p = tableMap.get(current.id);

      // Completion Time
      p.completion = time;

      // Turnaround Time
      // = Completion - Arrival
      p.turnaround = p.completion -
p.arrival;

      // Waiting Time
      // = Turnaround - Burst
      p.waiting = p.turnaround - p.burst;
    }
  }

  // ------------------------------------
  // Close final Gantt segment
  // ------------------------------------
  if (lastProcess !== null) {
    ganttChart.push({
      id: lastProcess,
      start: segmentStart,
      end: time,
      duration: time - segmentStart,
    });
  }

  // Convert Map → Array
  const table =
Array.from(tableMap.values());

  // ------------------------------------
  // Compute Average Waiting Time
  // ------------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  // ------------------------------------
  // Compute Average Turnaround Time
  // ------------------------------------
  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;

  // ------------------------------------
  // FINAL OUTPUT
  // ------------------------------------
  return {
    // Gantt Chart visualization data
    ganttChart,

    // Process scheduling metrics
    table,

    // Overall scheduling performance
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
```

```
    },
  };
}
```

priority.js

```
// ========================================
// PRIORITY SCHEDULING ALGORITHMS
// ========================================
//
// INCLUDED ALGORITHMS:
// 1. Priority Non-Preemptive Scheduling
//    - Once a process starts executing,
it continues until completion.
//    - CPU cannot interrupt the running
process.
// 2. Priority Preemptive Scheduling
//    - CPU can interrupt the current
process if a higher-priority
//      process arrives.
//    - Also known as Preemptive Priority
Scheduling.
//
// PRIORITY RULE:
// Lower priority number = Higher
priority
//    Priority 1 → Highest
//    Priority 5 → Lowest
//
// ========================================

// ========================================
// PRIORITY NON-PREEMPTIVE
// ========================================

export function
priorityNonPreemptive(processes) {
  // Total number of processes
  const n = processes.length;

  // Current CPU time
  let time = 0;

  // Counter for completed processes
  let completed = 0;

  // ------------------------------------
  // Clone processes and add completion
tracking
  // ------------------------------------
  const remaining = processes.map((p) =>
({
    ...p,
    completed: false,
  }));

  // Stores execution order for Gantt
Chart visualization
  const ganttChart = [];

  // Stores computed process metrics
  const tableMap = new Map();

  // ------------------------------------
  // Initialize table data
  // ------------------------------------
  processes.forEach((p) => {
    tableMap.set(p.id, {
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      priority: p.priority,
      completion: 0,
      waiting: 0,
      turnaround: 0,
    });
  });

  // ------------------------------------
  // MAIN SCHEDULING LOOP
  // Continues until all processes are
completed
  // ------------------------------------
  while (completed < n) {
    // Get all processes that already
arrived and not yet completed
    const available = remaining.filter(
      (p) => p.arrival <= time &&
!p.completed,
    );

    // ----------------------------------
    // CPU IDLE CONDITION
    // ----------------------------------
    // If no process is available, CPU
remains idle
    if (available.length === 0) {
      // Find next arriving process
      const nextArrival = Math.min(
        ...remaining.filter((p) =>
!p.completed).map((p) => p.arrival),
      );

      // Add idle segment to Gantt Chart
      ganttChart.push({
        id: "IDLE",
        start: time,
        end: nextArrival,
        duration: nextArrival - time,
      });

      // Move CPU time forward
      time = nextArrival;
      continue;
    }

    // ----------------------------------
    // Select highest-priority process
    // LOWER value = HIGHER priority
    // ----------------------------------
    available.sort((a, b) => {
      // If priorities are equal,
      // earlier arrival executes first
      if (a.priority === b.priority) {
        return a.arrival - b.arrival;
      }

      return a.priority - b.priority;
    });

    // Selected process
    const current = available[0];

    // ----------------------------------
    // Execute process completely
    // (NON-PREEMPTIVE)
    // ----------------------------------
    ganttChart.push({
      id: current.id,
      start: time,
      end: time + current.burst,
      duration: current.burst,
    });

    // Advance CPU time
    time += current.burst;

    // Mark process as completed
```

```
    current.completed = true;
    completed++;

    // ----------------------------------
    // Calculate scheduling metrics
    // ----------------------------------
    const p = tableMap.get(current.id);

    // Completion Time
    p.completion = time;

    // Turnaround Time = Completion -
Arrival
    p.turnaround = p.completion -
p.arrival;

    // Waiting Time = Turnaround - Burst
    p.waiting = p.turnaround - p.burst;
  }

  // Convert Map into array format
  const table =
Array.from(tableMap.values());

  // -----------------------------------
  // Compute averages
  // -----------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;

  // -----------------------------------
  // FINAL OUTPUT
  // -----------------------------------
  return {
    ganttChart,
    table,
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}

// ======================================
// PRIORITY PREEMPTIVE
// ======================================

export function
priorityPreemptive(processes) {
  // Total number of processes
  const n = processes.length;

  // Current CPU time
  let time = 0;

  // Completed process counter
  let completed = 0;

  // -----------------------------------
  // Clone processes and track remaining
burst time
  // -----------------------------------
  const remaining = processes.map((p) =>
({
    ...p,
    remaining: p.burst,
    completed: false,
  }));

  // Stores execution segments
  const ganttChart = [];

  // Stores process metrics
  const tableMap = new Map();

  // -----------------------------------
  // Initialize table
  // -----------------------------------
  processes.forEach((p) => {
    tableMap.set(p.id, {
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      priority: p.priority,
      completion: 0,
      waiting: 0,
      turnaround: 0,
    });
  });

  // Tracks currently executing process
  let lastProcess = null;

  // Start time of current Gantt segment
  let segmentStart = 0;

  // -----------------------------------
  // MAIN LOOP
  // Executes one time unit at a time
  // -----------------------------------
  while (completed < n) {
    // Get all arrived and unfinished
processes
    const available = remaining.filter(
      (p) => p.arrival <= time &&
!p.completed,
    );

    // ----------------------------------
    // CPU IDLE CONDITION
    // ----------------------------------
    if (available.length === 0) {
      // Save previous process segment
      if (lastProcess !== null) {
        ganttChart.push({
          id: lastProcess,
          start: segmentStart,
          end: time,
          duration: time - segmentStart,
        });

        lastProcess = null;
      }

      // Jump to next process arrival
      const nextArrival = Math.min(
        ...remaining.filter((p) =>
!p.completed).map((p) => p.arrival),
      );

      // Add idle segment
      ganttChart.push({
        id: "IDLE",
        start: time,
        end: nextArrival,
        duration: nextArrival - time,
      });

      time = nextArrival;
      continue;
    }

    // ----------------------------------
    // Select highest-priority process
    // ----------------------------------
    available.sort((a, b) => {
      // Tie breaker: earlier arrival
first
      if (a.priority === b.priority) {
        return a.arrival - b.arrival;
```

```
      }

      return a.priority - b.priority;
    });

    const current = available[0];

    // ----------------------------------
    // PROCESS SWITCH / PREEMPTION
    // ----------------------------------
    // If CPU switches to another
process,
    // save previous Gantt segment
    if (lastProcess !== current.id) {
      if (lastProcess !== null) {
        ganttChart.push({
          id: lastProcess,
          start: segmentStart,
          end: time,
          duration: time - segmentStart,
        });
      }

      // Start new segment
      lastProcess = current.id;
      segmentStart = time;
    }

    // ----------------------------------
    // Execute process for 1 time unit
    // ----------------------------------
    current.remaining--;
    time++;

    // ----------------------------------
    // Process Completion Check
    // ----------------------------------
    if (current.remaining === 0) {
      current.completed = true;
      completed++;

      const p = tableMap.get(current.id);

      // Completion Time
      p.completion = time;

      // Turnaround Time
      p.turnaround = p.completion -
p.arrival;

      // Waiting Time
      p.waiting = p.turnaround - p.burst;
    }
  }

  // ------------------------------------
  // Save final running segment
  // ------------------------------------
  if (lastProcess !== null) {
    ganttChart.push({
      id: lastProcess,
      start: segmentStart,
      end: time,
      duration: time - segmentStart,
    });
  }

  // Convert Map to array
  const table =
Array.from(tableMap.values());

  // ------------------------------------
  // Compute averages
  // ------------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;

  // ------------------------------------
  // FINAL OUTPUT
  // ------------------------------------
  return {
    ganttChart,
    table,
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}
```

priorityRR.js

```
// ======================================
// PRIORITY + ROUND ROBIN SCHEDULING
ALGORITHMS
// ======================================
//
// This algorithm combines:
// 1. Priority Scheduling
//    - Processes are grouped based on
priority.
//    - Lower priority number = Higher
priority.
// 2. Round Robin (RR)
//    - Processes within the same
priority level are executed
//      using a fixed Time Quantum.
//
// INCLUDED ALGORITHMS:
// 1. priorityRRNonPreemptive()
//    - Higher-priority queues execute
first.
//    - Once a queue starts executing, it
continues using RR.
//    - Running process is NOT
interrupted by higher-priority arrivals.
// 2. priorityRRPreemptive()
//    - Uses RR inside each priority
queue.
//    - Running process CAN be
interrupted if a higher-priority
//      process arrives.
//
// ======================================

// ======================================
// PRIORITY ROUND ROBIN - NON PREEMPTIVE
// ======================================

export function
priorityRRNonPreemptive(processes,
timeQuantum) {
  // Total number of processes
  const n = processes.length;

  // Current CPU time
  let time = 0;

  // Number of completed processes
  let completed = 0;

  // ------------------------------------
  // Normalize process list
  // Add remaining burst tracker
  // Sort by arrival time
  // ------------------------------------
```

```
  const remaining = processes
    .map((p) => ({
      ...p,
      remaining: p.burst,
      completed: false,
    }))
    .sort((a, b) => a.arrival -
b.arrival);

  // Stores execution timeline
  const ganttChart = [];

  // Stores process metrics
  const tableMap = new Map();

  // -------------------------------------
  // Initialize process table
  // -------------------------------------
  processes.forEach((p) => {
    tableMap.set(p.id, {
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      priority: p.priority,
      completion: 0,
      waiting: 0,
      turnaround: 0,
    });
  });

  // -------------------------------------
  // Queue structure:
  // priority number → array of processes
  //
  // Example:
  // queues = {
  //   1: [P1, P2],
  //   2: [P3]
  // }
  // -------------------------------------
  const queues = new Map();

  // Tracks next arriving process index
  let i = 0;

  // -------------------------------------
  // Helper function:
  // Adds idle time into Gantt Chart
  // -------------------------------------
  function pushIdle(start, end) {
    if (end <= start) return;

    const last =
ganttChart[ganttChart.length - 1];

    // Merge consecutive idle blocks
    if (last && last.id === "IDLE") {
      last.end = end;
      last.duration = last.end -
last.start;
      return;
    }

    ganttChart.push({
      id: "IDLE",
      start,
      end,
      duration: end - start,
    });
  }

  // -------------------------------------
  // MAIN SCHEDULING LOOP
  // -------------------------------------
  while (completed < n) {
    // ---------------------------------
    // Add newly arrived processes to
their priority queue
    // ---------------------------------
    while (i < n && remaining[i].arrival
<= time) {
      const p = remaining[i];

      // Create queue if not existing
      if (!queues.has(p.priority)) {
        queues.set(p.priority, []);
      }

      // Add process into its priority
queue
      queues.get(p.priority).push(p);

      i++;
    }

    // ---------------------------------
    // Find highest-priority queue with
available processes
    // ---------------------------------
    const available = [...queues.keys()]
      .filter((k) => queues.get(k).length
> 0)
      .sort((a, b) => a - b); // lower
number = higher priority

    // ---------------------------------
    // CPU IDLE CONDITION
    // ---------------------------------
    if (available.length === 0) {
      const nextArrival = i < n ?
remaining[i].arrival : undefined;

      if (nextArrival === undefined)
break;

      if (nextArrival > time) {
        pushIdle(time, nextArrival);
        time = nextArrival;
      }

      continue;
    }

    // Highest priority queue
    const priority = available[0];

    const queue = queues.get(priority);

    // Select next process using RR
    const current = queue.shift();

    const startTime = time;

    // ---------------------------------
    // ROUND ROBIN EXECUTION
    // Execute up to timeQuantum
    // ---------------------------------
    let execTime = 0;

    while (execTime < timeQuantum &&
current.remaining > 0) {
      current.remaining--;
      time++;
      execTime++;

      // -------------------------------
      // Add newly arrived processes
during execution
      // -------------------------------
      while (i < n &&
remaining[i].arrival <= time) {
        const p = remaining[i];

        if (!queues.has(p.priority)) {
```

```
          queues.set(p.priority, []);
        }

        queues.get(p.priority).push(p);

        i++;
      }
    }

    // ----------------------------------
    // Add execution segment to Gantt
Chart
    // ----------------------------------
    ganttChart.push({
      id: current.id,
      start: startTime,
      end: time,
      duration: time - startTime,
    });

    // ----------------------------------
    // Requeue unfinished process
    // OR finalize completed process
    // ----------------------------------
    if (current.remaining > 0) {
      // Put back at end of queue
      queue.push(current);
    } else {
      current.completed = true;
      completed++;

      const p = tableMap.get(current.id);

      // Completion Time
      p.completion = time;

      // Turnaround Time
      p.turnaround = p.completion -
p.arrival;

      // Waiting Time
      p.waiting = p.turnaround - p.burst;
    }
  }

  // Convert Map → Array
  const table =
Array.from(tableMap.values());

  // -------------------------------------
  // Compute averages
  // -------------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;

  // -------------------------------------
  // FINAL OUTPUT
  // -------------------------------------
  return {
    ganttChart,
    table,
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}

// ======================================
// PRIORITY ROUND ROBIN - PREEMPTIVE
// ======================================

export function
priorityRRPreemptive(processes,
timeQuantum) {
  // Total number of processes
  const n = processes.length;

  // Current CPU time
  let time = 0;

  // Completed process counter
  let completed = 0;

  // ------------------------------------
  // Clone process list
  // Add remaining burst tracking
  // Sort by arrival
  // ------------------------------------
  const remaining = processes
    .map((p) => ({
      ...p,
      remaining: p.burst,
      completed: false,
    }))
    .sort((a, b) => a.arrival -
b.arrival);

  // Stores execution segments
  const ganttChart = [];

  // Stores scheduling metrics
  const tableMap = new Map();

  // ------------------------------------
  // Initialize process table
  // ------------------------------------
  processes.forEach((p) => {
    tableMap.set(p.id, {
      id: p.id,
      arrival: p.arrival,
      burst: p.burst,
      priority: p.priority,
      completion: 0,
      waiting: 0,
      turnaround: 0,
    });
  });

  // Priority queues
  const queues = new Map();

  // Arrival tracker
  let i = 0;

  // ------------------------------------
  // Helper function:
  // Adds idle CPU segment
  // ------------------------------------
  function pushIdle(start, end) {
    if (end <= start) return;

    const last =
ganttChart[ganttChart.length - 1];

    // Merge consecutive idle blocks
    if (last && last.id === "IDLE") {
      last.end = end;
      last.duration = last.end -
last.start;
      return;
    }

    ganttChart.push({
      id: "IDLE",
      start,
      end,
      duration: end - start,
    });
  }
```

```
  // ------------------------------------
  // MAIN LOOP
  // ------------------------------------
  while (completed < n) {
    // ----------------------------------
    // Add arriving processes
    // ----------------------------------
    while (i < n && remaining[i].arrival
<= time) {
      const p = remaining[i];

      if (!queues.has(p.priority)) {
        queues.set(p.priority, []);
      }

      queues.get(p.priority).push(p);

      i++;
    }

    // ----------------------------------
    // Find highest-priority queue
    // ----------------------------------
    const available = [...queues.keys()]
      .filter((k) => queues.get(k).length
> 0)
      .sort((a, b) => a - b);

    // ----------------------------------
    // CPU IDLE CONDITION
    // ----------------------------------
    if (available.length === 0) {
      const nextArrival = i < n ?
remaining[i].arrival : undefined;

      if (nextArrival === undefined)
break;

      if (nextArrival > time) {
        pushIdle(time, nextArrival);
        time = nextArrival;
      }

      continue;
    }

    // Current highest priority
    const priority = available[0];

    const queue = queues.get(priority);

    // Get next RR process
    const current = queue.shift();

    let startTime = time;

    let execTime = 0;

    // ----------------------------------
    // Execute process using Round Robin
    // ----------------------------------
    while (execTime < timeQuantum &&
current.remaining > 0) {
      current.remaining--;
      time++;
      execTime++;

      // --------------------------------
      // Add newly arrived processes
      // --------------------------------
      while (i < n &&
remaining[i].arrival <= time) {
        const p = remaining[i];

        if (!queues.has(p.priority)) {
          queues.set(p.priority, []);
        }

        queues.get(p.priority).push(p);

        i++;
      }

      // --------------------------------
      // PREEMPTION CHECK
      // --------------------------------
      // Interrupt current process if a
higher-priority queue exists
      const higherExists =
[...queues.keys()].some(
        (k) => k < priority &&
queues.get(k).length > 0,
      );

      if (higherExists) {
        // Stop execution immediately
        break;
      }
    }

    // ----------------------------------
    // Save execution segment
    // ----------------------------------
    ganttChart.push({
      id: current.id,
      start: startTime,
      end: time,
      duration: time - startTime,
    });

    // ----------------------------------
    // Requeue unfinished process
    // OR finalize completed process
    // ----------------------------------
    if (current.remaining > 0) {
      if (!queues.has(priority)) {
        queues.set(priority, []);
      }

      queues.get(priority).push(current);
    } else {
      current.completed = true;
      completed++;

      const p = tableMap.get(current.id);

      // Completion Time
      p.completion = time;

      // Turnaround Time
      p.turnaround = p.completion -
p.arrival;

      // Waiting Time
      p.waiting = p.turnaround - p.burst;
    }
  }

  // Convert Map → Array
  const table =
Array.from(tableMap.values());

  // ------------------------------------
  // Compute averages
  // ------------------------------------
  const avgWaiting = table.reduce((sum,
p) => sum + p.waiting, 0) / n;

  const avgTurnaround =
table.reduce((sum, p) => sum +
p.turnaround, 0) / n;

  // ------------------------------------
```

```
  // FINAL OUTPUT
  // ------------------------------------
  return {
    ganttChart,
    table,
    averages: {
      waiting: avgWaiting,
      turnaround: avgTurnaround,
    },
  };
}
```

getInput.js

```
// ========================================
// INPUT TABLE MANAGEMENT + DATA
EXTRACTION
// ========================================
//
// This module handles:
// 1. Dynamic row addition/removal in the
process input table
// 2. Parsing user input into process
objects
// 3. Retrieving time quantum for Round
Robin-based algorithms
//
// It acts as the "data collection layer"
between the UI and scheduler
// algorithms.
//
// ========================================

// ========================================
// DOM ELEMENT REFERENCES
// ========================================
const tableBody =
document.getElementById("table-body");
const addBtn =
document.getElementById("add-row-btn");
const removeBtn =
document.getElementById("remove-row-
btn");

// ========================================
// ADD NEW PROCESS ROW
// ========================================
// Creates a new table row dynamically
when "+" button is clicked.
// Each row represents a process with:
// - Process ID (P1, P2, ...)
// - Arrival Time input
// - Burst Time input
// - Priority input (hidden unless
needed)
// ========================================
addBtn.addEventListener("click", () => {
  // Determine next process number based
on current row count
  const rowCount = tableBody.rows.length
+ 1;

  // Create new table row
  const row =
document.createElement("tr");

  // Inject input fields into row
  row.innerHTML = `
    <td>P${rowCount}</td>
    <td><input type="number"
placeholder="0" min="0"></td>
    <td><input type="number"
placeholder="1" min="1"></td>
    <td class="prio-col hide-col">
      <input type="number"
placeholder="0" min="0">
    </td>
  `;

  // ------------------------------------
  // Show priority column only if
algorithm uses priority scheduling
  // ------------------------------------
  if (document.getElementById("sched-
algo").value.includes("priority")) {
    row.querySelector(".prio-
col").classList.remove("hide-col");
  }

  // Append row to table body
  tableBody.appendChild(row);
});

// ========================================
// REMOVE LAST PROCESS ROW
// ========================================
// Ensures at least 3 processes remain
(P1-P3 default minimum)
removeBtn.addEventListener("click", () =>
{
  if (tableBody.rows.length > 3) {
    tableBody.deleteRow(-1);
  }
});

// ========================================
// HELPER: INPUT NORMALIZATION
// ========================================
// Converts empty input values into 0 to
avoid NaN issues
function parseOrZero(value) {
  return value === "" ? 0 :
parseInt(value);
}

// ========================================
// EXTRACT PROCESS INPUT DATA
// ========================================
// Reads table inputs and converts them
into an array of objects
//
// OUTPUT FORMAT:
// [
//   { id: "P1", arrival: 0, burst: 5,
priority: 2 }
// ]
// ========================================
export function getInput() {
  const rows =
document.querySelectorAll("#table-body
tr");

  const processes = [];

  rows.forEach((row, index) => {
    const cols =
row.querySelectorAll("td");

    // Extract raw input values
    const arrivalInput =
cols[1].querySelector("input").value;
    const burstInput =
cols[2].querySelector("input").value;
    const priorityInput =
cols[3].querySelector("input").value;

    // Normalize values (empty → 0)
    const arrival =
parseOrZero(arrivalInput);
    const burst =
```

```
parseOrZero(burstInput);
    const priority =
parseOrZero(priorityInput);

    // Build process object
    processes.push({
      id: `P${index + 1}`,
      arrival,
      burst,
      priority,
    });
  });

  return processes;
}

// ========================================
// GET TIME QUANTUM (ROUND ROBIN)
// ========================================
// Validates user input for time quantum
//
// RULE:
// - Must be a number
// - Must be > 0
// ========================================
export function getTimeQuantum() {
  const tqInput =
document.getElementById("time-quantum");

  const value = parseInt(tqInput.value);

  // Validation check
  if (isNaN(value) || value <= 0) {
    alert("Time Quantum must be greater
than 0.");
    return null;
  }

  return value;
}
```

validateInput.js

```
// ========================================
// INPUT VALIDATION MODULE
// ========================================
//
// This function ensures that all process
inputs are valid before
// being passed into any CPU scheduling
algorithm.
//
// It acts as a SAFETY CHECK layer to
prevent:
// • Invalid burst times
// • Negative or zero execution values
// • Incorrect scheduling computation
results
// ========================================

// ========================================
// VALIDATE PROCESS INPUTS
// ========================================
// Ensures that each process has a valid
burst time (> 0)
//
// INPUT FORMAT:
// processes = [
//   { id, arrival, burst, priority }
// ]
//
// OUTPUT FORMAT:
// true  → if all inputs are valid
// false → if at least one invalid input
is found
// ========================================
export function validateInput(processes)
{
  // Loop through all processes
  for (let p of processes) {
    // ----------------------------------
    // INVALID BURST TIME CHECK
    // ----------------------------------
    // Burst time represents CPU
execution time.
    // It must be greater than 0 to be
meaningful in scheduling.
    // ----------------------------------
    if (p.burst <= 0) {
      alert(`Process ${p.id}: Burst time
must be greater than 0`);

      return false;
    }
  }

  // If all processes are valid, allow
scheduling to proceed
  return true;
}
```

renderTable.js

```
// ========================================
// RESULT TABLE RENDERING MODULE
// ========================================
//
// This module displays the computed
scheduling results inside the
// Process Scheduling Table section of
the application.
//
// It dynamically generates table rows
based on the selected
// scheduling algorithm's output.
//
// ========================================

// ========================================
// RENDER RESULT TABLE
// ========================================
// Displays:
// • Process information
// • Completion Time
// • Waiting Time
// • Turnaround Time
// • Average metrics
//
// INPUT FORMAT:
// results = {
//   table: [...],
//   averages: { waiting, turnaround }
// }
// ========================================
export function renderTable(results) {
  // Reference to result table body
  const tbody =
document.querySelector("#result-table
tbody");

  // ------------------------------------
  // Clear previously rendered results
  // ------------------------------------
  tbody.innerHTML = "";

  // ====================================
  // RENDER PROCESS ROWS
  // ====================================
  results.table.forEach((p) => {
```

```
    // Create new row for each process
    const row =
document.createElement("tr");

    // ----------------------------------
    // Populate process scheduling data
    // ----------------------------------
    row.innerHTML = `
      <td>${p.id}</td>
      <td>${p.arrival}</td>
      <td>${p.burst}</td>

      <!-- Show priority value if
available -->
      <td>${p.priority ?? "-"}</td>

      <td>${p.completion}</td>
      <td>${p.waiting}</td>
      <td>${p.turnaround}</td>
    `;

    // Add row to table
    tbody.appendChild(row);
  });

  // ====================================
  // RENDER AVERAGE METRICS ROW
  // ====================================
  // Displays:
  // • Average Waiting Time
  // • Average Turnaround Time
  // ====================================
  const metricsRow =
document.createElement("tr");

  metricsRow.id = "metrics";

  metricsRow.innerHTML = `
    <td colspan="5">Average</td>

    <!-- Average Waiting Time -->
    <td>${results.averages.waiting.toFixe
d(2)}</td>

    <!-- Average Turnaround Time -->
    <td>${results.averages.turnaround.toF
ixed(2)}</td>
  `;

  // Append metrics row to table
  tbody.appendChild(metricsRow);
}
```

renderGantt.js

```
// ======================================
// GANTT CHART RENDERING MODULE
// ======================================
//
// This module is responsible for
visually rendering the CPU scheduling
// execution timeline (Gantt chart).
//
// It transforms algorithm output into a
proportional UI representation
// where each process block reflects its
execution duration.
//
// ======================================

import { getColor } from
"../utils/processColors.js";

// ======================================
// INSERT IDLE SEGMENTS
// ======================================
// Ensures that gaps between process
executions are visually shown
// as "IDLE" blocks in the Gantt chart.
//
// This improves accuracy of the timeline
representation.
//
// INPUT:
// ganttChart = [
//   { id, start, end, duration }
// ]
//
// OUTPUT:
// Modified array including IDLE segments
where gaps exist
// ======================================
function addIdleSegments(ganttChart) {
  const result = [];

  for (let i = 0; i < ganttChart.length;
i++) {
    const current = ganttChart[i];

    // First process has no previous
comparison
    if (i === 0) {
      result.push(current);
      continue;
    }

    const prev = ganttChart[i - 1];

    // ----------------------------------
    // Detect CPU idle time (gap between
processes)
    // ----------------------------------
    if (current.start > prev.end) {
      result.push({
        id: "IDLE",
        start: prev.end,
        end: current.start,
        duration: current.start -
prev.end,
      });
    }

    result.push(current);
  }

  return result;
}

// ======================================
// RENDER GANTT CHART
// ======================================
// Converts scheduling output into a
visual bar-based timeline.
//
// Each process is rendered as a
proportional block where:
// - Width is based on execution duration
// - Color represents process identity
// - IDLE time is visually distinct
//
// INPUT:
// ganttChart = [
//   { id, start, end, duration }
// ]
// ======================================
export function renderGantt(ganttChart) {
  // DOM containers
  const container =
document.getElementById("gantt-bar-
container");
  const timeline =
```

```
document.getElementById("gantt-
timeline");

  // Clear previous render
  container.innerHTML = "";
  timeline.innerHTML = "";

  // Add IDLE segments for accurate
timeline visualization
  const chart =
addIdleSegments(ganttChart);

  // ====================================
  // RENDER GANTT BARS
  // ====================================
  chart.forEach((item) => {
    const block =
document.createElement("div");

    block.classList.add("gantt-block");

    // ----------------------------------
    // Proportional sizing based on
execution duration
    // ----------------------------------
    block.style.flexGrow = item.duration;
    block.style.flexBasis = "0%";
    block.style.flexShrink = "1";

    block.textContent = item.id;

    // ----------------------------------
    // Styling for IDLE blocks
    // ----------------------------------
    if (item.id === "IDLE") {
      block.style.background = "#fafafa";
      block.style.color = "#777";
      block.style.fontStyle = "italic";
      block.style.fontSize = "0.9rem";
    } else {
      // Assign consistent color per
process
      block.style.background =
getColor(item.id);
    }

    container.appendChild(block);
  });

  // ====================================
  // RENDER TIMELINE LABELS (START TIMES)
  // ====================================
  chart.forEach((item) => {
    const span =
document.createElement("span");

    span.textContent = item.start;

    // Align labels with corresponding
blocks
    span.style.flexGrow = item.duration;
    span.style.flexBasis = "0%";
    span.style.flexShrink = "1";

    span.style.textAlign = "left";

    timeline.appendChild(span);
  });

  // ====================================
  // FINAL END TIME MARKER
  // ====================================
  const end =
document.createElement("span");

  end.textContent = chart[chart.length -
1].end;

  // Prevent distortion of flex scaling
  end.style.flex = "0 0 auto";

  timeline.appendChild(end);
}
```

simulateGantt.js

```
// ======================================
// GANTT CHART SIMULATION MODULE
// ======================================
//
// This module animates the execution of
CPU scheduling algorithms
// using a dynamic Gantt chart
visualization.
//
// Instead of rendering the entire chart
instantly,
// blocks are displayed progressively
based on execution duration.
//
// This creates a simulation effect that
visually demonstrates:
// • Process execution order
// • CPU idle time
// • Scheduling behavior over time
//
// ======================================

import { getColor } from
"../utils/processColors.js";

// ======================================
// SIMULATION STATE FLAG
// ======================================
// Prevents multiple simulations from
running simultaneously
let isSimulating = false;

// ======================================
// INSERT IDLE SEGMENTS
// ======================================
// Detects gaps between process
executions and inserts
// "IDLE" segments into the Gantt chart.
//
// This ensures accurate CPU timeline
visualization.
//
// INPUT:
// ganttChart = [
//   { id, start, end, duration }
// ]
//
// OUTPUT:
// Updated chart including IDLE blocks
// ======================================
function addIdleSegments(ganttChart) {
  const result = [];

  for (let i = 0; i < ganttChart.length;
i++) {
    const current = ganttChart[i];

    // First segment has no previous
comparison
    if (i === 0) {
      result.push(current);
      continue;
    }
```

```
    const prev = ganttChart[i - 1];

    // ----------------------------------
    // Detect CPU idle gap
    // ----------------------------------
    if (current.start > prev.end) {
      result.push({
        id: "IDLE",
        start: prev.end,
        end: current.start,
        duration: current.start -
prev.end,
      });
    }

    result.push(current);
  }

  return result;
}

// ======================================
// SIMULATE GANTT CHART
// ======================================
// Animates process execution blocks one
at a time.
//
// FUNCTIONALITY:
// • Clears old chart
// • Adds blocks progressively
// • Waits based on process duration
// • Displays execution timeline
// ======================================
export async function
simulateGantt(ganttChart, speed = 750) {
  // Prevent overlapping simulations
  if (isSimulating) return;

  isSimulating = true;

  // DOM containers
  const container =
document.getElementById("gantt-bar-
container");
  const timeline =
document.getElementById("gantt-
timeline");

  // Clear previous simulation
  container.innerHTML = "";
  timeline.innerHTML = "";

  // Add idle segments for accurate
visualization
  const chart =
addIdleSegments(ganttChart);

  // ======================================
  // PROCESS EACH GANTT SEGMENT
  // ======================================
  for (let i = 0; i < chart.length; i++)
{
    const item = chart[i];

    // ==================================
    // CREATE GANTT BLOCK
    // ==================================
    const block =
document.createElement("div");

    block.classList.add("gantt-block");

    // ------------------------
    // Scale block width proportionally
to execution duration
    // ----------------------------------
    block.style.flexGrow = item.duration;
    block.style.flexBasis = "0%";
    block.style.flexShrink = "1";

    // Display process ID
    block.textContent = item.id;

    // ==================================
    // APPLY BLOCK STYLING
    // ==================================
    if (item.id === "IDLE") {
      // CPU idle styling
      block.style.background = "#fafafa";
      block.style.color = "#777";
      block.style.fontStyle = "italic";
    } else {
      // Assign consistent process color
      block.style.background =
getColor(item.id);
    }

    // Add block to chart container
    container.appendChild(block);

    // ==================================
    // CREATE TIMELINE LABEL
    // ==================================
    const span =
document.createElement("span");

    // Display process start time
    span.textContent = item.start;

    // Match proportional scaling with
block
    span.style.flexGrow = item.duration;
    span.style.flexBasis = "0%";
    span.style.flexShrink = "1";

    span.style.textAlign = "left";

    // Add label to timeline
    timeline.appendChild(span);

    // ==================================
    // ANIMATION DELAY
    // ==================================
    // Wait based on execution duration
    //
    // Longer processes appear longer on
screen.
    // ----------------------------------
    await new Promise((resolve) =>
setTimeout(resolve, item.duration *
speed));
  }

  // ======================================
  // DISPLAY FINAL END TIME
  // ======================================
  const end =
document.createElement("span");

  end.textContent = chart[chart.length -
1].end;

  // Prevent flex distortion
  end.style.flex = "0 0 auto";

  timeline.appendChild(end);

  // ======================================
  // RESET SIMULATION STATE
  // ======================================
  isSimulating = false;
}
```

processColors.js

```
// ====================================
// PROCESS COLOR UTILITY MODULE
// ====================================
//
// This module generates and stores
consistent colors for processes
// used in the Gantt chart
visualization.
//
// PURPOSE:
// • Each process receives a unique
color
// • Colors remain consistent across
renders/simulations
// • Improves readability of scheduling
visualization
//
// ====================================

// Stores generated colors
const processColors = {};

// Retro-inspired HSL palette
const retroPalette = [
  [180, 35, 45], // teal
  [30, 50, 45], // amber
  [260, 35, 48], // violet
  [140, 35, 42], // green
  [210, 30, 48], // slate blue
  [10, 45, 45], // coral
  [50, 65, 40], // golden yellow
];

// Normalizes process IDs for
consistency
function normalizeId(id) {
  return
String(id).trim().toUpperCase();
}

// Returns a consistent color for a
process ID
export function getColor(processId) {
  const id = normalizeId(processId);

  // Return cached color if available
  if (processColors[id]) return
processColors[id];

  // Generate hash from process ID
  let hash = 0;

  for (let i = 0; i < id.length; i++) {
    hash = id.charCodeAt(i) + ((hash <<
5) - hash);
  }

  // Select color from palette
  const index = Math.abs(hash) %
retroPalette.length;

  const [h, s, l] =
retroPalette[index];

  // Small brightness variation
  const lightnessOffset =
(Math.abs(hash) % 10) - 8;

  // Final HSL color
  const color = `hsl(${h}, ${s}%, ${l +
lightnessOffset}%)`;

  // Cache generated color
  processColors[id] = color;

  return color;
}
```